BANK MANAGEMENT SYSTEM USING ZOHO CREATOR

A PROJECT REPORT

Submitted by

HARISH M

(241501066)

in partial fulfilment of the End Semester Examination for the course

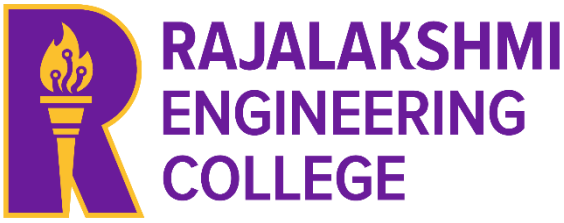
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**Bank Management System using Zoho Creator**" is the bonafide work of **Harish M (241501066),** who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year **2026–2027**, **Semester V**, under my supervision.

SIGNATURE

Dr. M. Ayyadurai
HEAD OF THE DEPARTMENT
Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Mrs. M. Priya, M.E.
COURSE FACULTY
Assistant Professor
Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, **Harish M (241501066),** hereby declare that the project report titled "**Bank Management System using Zoho Creator",** submitted for the End Semester Examination of the course **CB23F35 – Low-Code Application Development using Zoho Creator**, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date:

SIGNATURE OF THE STUDENT
Harish M
241501066

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Mr. S. Meganathan and Chairperson Dr. (Mrs.) Thangam Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S.N. Murugesan, and our Vice Principal, Dr. S.P. Srinivasan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. M. Ayyadurai, Head of the Department, Department of Artificial Intelligence and Machine Learning, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Mrs. M. Priya, M.E., Assistant Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

HARISH M

ABSTRACT

Many small bank branches still open accounts on paper and keep balances, deposits and loan records in spreadsheets. Staff type the same customer details many times, managers sign approvals by hand, and there is no single place to see the total balance of the branch. This leads to delays, errors in balances and weak control over who can see customer data.

This project builds a Bank Management System on the Zoho Creator low-code platform. The application has four forms (Branch Master, Account Opening, Transaction and Loan Application) linked through lookup fields. The Account Opening form uses 27 different field types, including an auto-generated account number, a formula for yearly interest, a QR code, a signature and file uploads. Deluge scripts set the opening balance, block withdrawals that exceed the balance, update the account balance on every transaction and calculate the monthly EMI of a loan using the reducing-balance formula. A daily schedule emails fixed-deposit maturity reminders, an approval process sends loans above ₹5,00,000 to the branch manager, and a blueprint moves each account through the stages Applied, KYC Verified, Active and Closed. List, Kanban and Calendar reports and a dashboard page give the manager a live view of accounts and balances.

The application was tested on the live Zoho Creator app with seven customer accounts, two transactions and two loans. All key rules worked as expected: the balance check stopped an invalid withdrawal, the EMI of ₹6,00,000 at 10% for 12 months was calculated as ₹52,749.53, and a ₹7,50,000 loan was routed for approval and marked Approved. The work shows that a working banking prototype can be built in days instead of months, with less code and built-in hosting.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Banking; Loan EMI*

PROJECT AT A GLANCE

Attribute	Details
Project Title	Bank Management System using Zoho Creator
Application Domain	Finance / Banking (branch operations)
Student Name & Register No.	Harish M – 241501066
Zoho Creator Edition	Zoho Creator trial (15-day trial plan)
Application Name (Link Name)	Bank Management System (bank-management-system)
Published / Live URL	https://creatorapp.zoho.in/harish.m.2024.aiml_rajala578/bank-management-system
Demo Credentials (per role)	Admin: account owner login Bank Staff: Add accounts, Transactions etc.
Number of Forms / Subforms	4 / 1 (Nominee Details)
Number of Reports (by type)	List: 4 Kanban: 1 Calendar: 1
Number of Pages / Dashboards	1 (Bank Command Center)
Number of Workflows	Form: 4 Schedule: 1 Approval: 1 Blueprint: 1 Batch: 1
Deluge Functions / Custom Actions	2 functions (Load_Sample_Data, Get_Exchange_Rates); 1 pro-code widget (EMI Calculator)
Integrations	Zoho Mail (send mail); Frankfurter REST API (invoke URL)
Roles & Profiles Defined	Administrator (owner); Read and Write profiles; Customer portal profile with own-records access
Environments Used	Production only (environments not enabled on the trial)
Date of Production Deployment	16 October 2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xii
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development 4
2.4 Literature Survey 5
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 6
REQUIREMENT ANALYSIS 7
3.1 Stakeholders and User Roles 7
3.2 Functional Requirements 7
3.3 Non-Functional Requirements 9
3.4 Hardware and Software Requirements 9
3.5 Use Case Model 9
SYSTEM DESIGN 12
4.1 System Architecture 12
4.2 Data Model 12
4.3 Form Design 13
4.4 Process and Workflow Design 16
4.5 User Interface and Navigation Design 17
4.6 Access Control Design 18
IMPLEMENTATION 19
5.1 Application Setup 19
5.2 Forms and Data Capture 19

5.3 Reports 23
5.4 Pages and Dashboards 25
5.5 Deluge Scripting 26
5.6 Workflows and Automation 29
5.7 Integrations 32
5.8 User Management and Role-Based Access 34
5.9 AI and Analytics Features 35
5.10 Pro-Code Extension 35
5.11 Mobile and Portal Access 37
TESTING AND DEBUGGING 38
6.1 Testing Strategy 38
6.2 Test Cases 38
6.3 Debugging Deluge Scripts 39
6.4 Defect Log 42
6.5 User Acceptance Testing 43
DEPLOYMENT, SECURITY AND GOVERNANCE 44
7.1 Environments and Deployment Strategy 44
7.2 Performance Optimisation 45
7.3 Security Features 45
7.4 Governance and Administration 45
RESULTS AND DISCUSSION 47
8.1 Application Output 47
8.2 Achievement of Objectives 48
8.3 Comparison with the Existing System 48
8.4 Limitations of the Low-Code Approach 49
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 49
CONCLUSION AND FUTURE ENHANCEMENTS 50
9.1 Conclusion 50
9.2 Future Enhancements 50
9.3 Learning Outcomes and Career Relevance 50
REFERENCES 51
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 52
APPENDIX B – COURSE OUTCOME MAPPING 55
APPENDIX C – COMPLETE DELUGE SOURCE CODE 56
APPENDIX D – USER MANUAL 62
D.1 Accessing the Application 62
D.2 Instructions for Branch Manager (Administrator) 62

D.3 Instructions for Bank Staff 62
APPENDIX E – EVALUATION RUBRIC 63
APPENDIX F – VIVA-VOCE RECORD SHEET 65

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application....4
Table 2.2: Summary of Literature Survey....5
Table 3.1: Stakeholders and User Roles....7
Table 3.2: Functional Requirements....7
Table 3.3: Non-Functional Requirements....9
Table 3.4: Hardware and Software Requirements....9
Table 3.5: Use Case Description – Record a Withdrawal....10
Table 4.1: Entities and Relationships....13
Table 4.2: Field Specification – Account Opening Form....13
Table 4.3: Field Specification – Branch Master Form....15
Table 4.4: Field Specification – Transaction Form....15
Table 4.5: Field Specification – Loan Application Form....16
Table 4.6: Role–Permission Matrix....18
Table 5.1: Reports Implemented....23
Table 5.2: Deluge Scripts Implemented....26
Table 5.3: Workflows Implemented....29
Table 5.4: Integrations Implemented....32
Table 6.1: Test Cases....38
Table 6.2: Defect Log....42
Table 6.3: User Acceptance Feedback....43
Table 7.1: Deployment Checklist....44
Table 7.2: Performance Optimisation Results....45
Table 7.3: Security Controls Implemented....45
Table 8.1: Mapping of Objectives to Outcomes....48
Table 8.2: Existing System versus Proposed Application....48
Table A.1: Feature Coverage Checklist....52
Table B.1: Mapping of Course Outcomes to Report Chapters....55
Table E.1: Evaluation Rubric for the Individual Project....63
Table E.2: Performance Level Descriptors....64
Table F.1: Viva-Voce Questions and Responses....65
Table F.2: Marks Summary....65

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 4
Figure 3.1: Use Case Diagram of the Bank Management System 10
Figure 4.1: System Architecture of the Bank Management System 12
Figure 4.2: Entity–Relationship Diagram (Zoho Creator Schema Builder) 12
Figure 4.3: Workflow / Process Flow for Recording a Transaction 17
Figure 4.4: Navigation Structure of the Bank Management System 18
Figure 5.1: Create New Application Window in Zoho Creator 19
Figure 5.2: Application Home with Navigation Sections (Live Mode) 19
Figure 5.3: Account Opening Form in the Form Builder 20
Figure 5.4: Transaction Form in the Form Builder 21
Figure 5.5: Loan Application Form in the Form Builder 21
Figure 5.6: Account Opening Form in Live Mode (Address, PAN, Deposit) 21
Figure 5.7: Account Opening Form in Live Mode (Signature, Formula and Status) 22
Figure 5.8: Nominee Details Subform in the Account Opening Form (Live Mode) 22
Figure 5.9: Account Record with QR Code, Signature and Nominee Details 22
Figure 5.10: Report Types Available in Zoho Creator 23
Figure 5.11: Account Status Kanban Report 24
Figure 5.12: FD Maturity Calendar Report 24
Figure 5.13: All Transactions Report 24
Figure 5.14: All Loan Applications Report with Calculated EMI 25
Figure 5.15: Pivot Chart Requires Zoho Analytics Configuration 25
Figure 5.16: Bank Command Center Page in the Page Builder 26
Figure 5.17: Bank Command Center Dashboard in Live Mode 26
Figure 5.18: Form Workflows Configured for the Account Opening Form 30
Figure 5.19: Deluge Editor with the Check and Update Balance Script 30
Figure 5.20: High Value Loan Approval Configuration 31
Figure 5.21: Account Lifecycle Blueprint with Stages and Transitions 31
Figure 5.22: Batch Workflow for Monthly Savings Interest Credit 31
Figure 5.23: Loan of ₹7,50,000 Waiting in Pending Tasks 32
Figure 5.24: Approval Summary with Approve and Reject Buttons 32
Figure 5.25: Schedule Script Executed: “FD reminders sent: 2” 33
Figure 5.26: REST API Call with invokeurl – Execution Log 34
Figure 5.27: User Permissions – Read and Write Profiles 34
Figure 5.28: Customer Portal Profile with Record-Level Access 35

Figure 5.29: Analytics on the Dashboard: Total Balance and Balance by Account Type 35
Figure 5.30: Pro-code Widget Uploaded in Settings → Widgets 36
Figure 5.31: EMI Calculator Widget on the Bank Command Center Page (Live Mode) 36
Figure 5.32: Phone Layout of a Form in the Design Tab 37
Figure 6.1: Deluge Execute Output with Info Log Message 40
Figure 6.2: Account Confirmation Email Received in the Inbox (DS-02) 40
Figure 6.3: FD Maturity Reminder Email Received in the Inbox (DS-05) 41
Figure 6.4: Blueprint Run: Applied → KYC Verified → Active 42
Figure 7.1: Application Settings (Administration) 46
Figure 8.1: Scenario Step 1 – Opening a New Account 47
Figure 8.2: Scenario Step 2 – Withdrawal Blocked for Insufficient Balance 47
Figure 8.3: Scenario Step 3 – Loan Approved with Calculated EMI 48

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
EMI	Equated Monthly Instalment
ER	Entity–Relationship
FD	Fixed Deposit
FR	Functional Requirement
IFSC	Indian Financial System Code
KYC	Know Your Customer
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
PAN	Permanent Account Number
QR	Quick Response (code)
RBAC	Role-Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Banking is one of the most record-heavy services in daily life. Every branch opens new accounts, accepts deposits and withdrawals, gives loans and keeps track of fixed deposits that mature on different dates. Each of these steps needs correct data, quick approval and a clear record of who did what.

This project uses a realistic case: a small co-operative bank branch in Chennai with three branch offices (Thandalam, Poonamallee and T Nagar), a branch manager, a few cashiers and loan officers, and a few hundred customers. Today the branch uses paper account-opening forms, an Excel sheet for balances and a register for loans. The manager signs every large loan on paper and prepares monthly summaries by hand.

The aim of this project is to move this whole process into one web application built on Zoho Creator, so that staff enter data once, rules are checked automatically, and the manager sees live numbers on a dashboard.

1.2 Problem Statement

Branch staff record accounts, transactions and loans in separate paper forms and spreadsheets, so data is repeated, balances are updated by hand, large loans wait for manual sign-off, and the manager has no live view of the branch.

- Customer details are typed again in the account register, the Excel balance sheet and the loan register.
- Balances are updated by hand, so a withdrawal larger than the balance can be recorded by mistake.
- Loan EMI is worked out on a calculator, which is slow and error-prone.
- Fixed deposits that are about to mature are missed because no one is reminded.
- Anyone with access to the Excel file can see and change every customer’s data.

1.3 Objectives

- To design a linked data model and four forms for branches, accounts, transactions and loans.

- To automate balance updates, withdrawal checks and EMI calculation using Deluge scripts.
- To route high-value loans for manager approval and track each account through a blueprint lifecycle.
- To send automatic email alerts, including daily fixed-deposit maturity reminders.
- To give the branch manager List, Kanban and Calendar reports and a live dashboard.

1.4 Scope of the Project

In scope:

- Branch master data, customer account opening with KYC document upload, deposits, withdrawals and fund transfers.
- Loan applications with automatic EMI, approval of loans above ₹5,00,000, and account lifecycle stages.
- Email notifications, daily FD maturity reminders, reports and a dashboard for the branch manager.
- Web access for the administrator and staff through the Zoho Creator app.

Out of scope: real money movement, links to core banking or UPI/NEFT, interest posting, cheque clearing, a customer self-service portal and the mobile app. These are listed as future work in Chapter 9.

1.5 Justification for a Low-Code Approach

Zoho Creator suits this problem because the branch needs forms, rules and reports more than custom code. Forms, reports and hosting are built in, so a working app can be made in days. Business rules such as balance checks and EMI are written in short Deluge scripts, and approvals, schedules and blueprints are set up through screens instead of code [2], [3]. Low-code platforms are designed to let a small team build and change business apps quickly [1], [6]. Security features such as login, permission profiles and audit history come with the platform, which lowers cost for a small branch.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5

details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the existing system a customer fills a paper form at the branch. A clerk checks the KYC copies, types the details into an Excel ledger and files the paper. The manager signs the form and any large loan. Deposits and withdrawals are written in a register and the Excel balance is changed by hand. At month end, staff count records and prepare summaries for the manager. Figure 2.1 shows this flow.

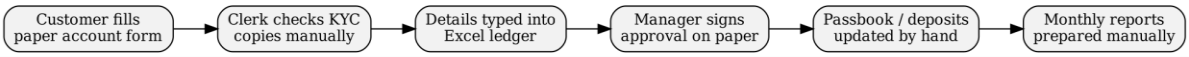


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- The same data is typed many times, which wastes time and causes mismatches.
- There is no automatic check, so balances can go negative and EMI can be wrong.
- Approvals are on paper, so loans wait when the manager is busy and there is no record of when they were approved.
- There is no reminder system for maturing fixed deposits.
- There is no role-based access; the Excel file is open to everyone who has it.
- Reports are prepared by hand and are out of date as soon as they are made.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	About 10–12 weeks for forms, database, logic, login and hosting	About 1 week (built and tested in two working sessions)
Skills required	Java/Python, SQL, HTML/CSS/JS, server setup	Zoho Creator builder and basic Deluge scripting
Hosting & infrastructure	Own server or cloud, database and backups to manage	Hosted and backed up by Zoho
Mobile application	Separate Android/iOS app needed	Same app runs in the Zoho Creator mobile app

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Security & access control	Login, roles and encryption must be coded	Built-in login, permission profiles and roles
Maintenance & change	Code changes, testing and redeploy	Change a field or rule in the builder and save
Customisation limits	No limits	Limited to platform features; some need paid add-ons (e.g., pivot reports need Zoho Analytics)
Cost	High developer and hosting cost	Subscription per user; trial used for this project

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	W. Rosam and O. Kalnichevski, 2020 [1]	Low-Code Development: A Practical Guide	Explains what low-code is, its building blocks and where it fits	Basis for choosing a low-code approach for the branch
2	A. Sahay et al., 2020 [6]	Supporting the understanding and comparison of low-code development platforms	Compares common low-code platforms on features such as data modelling, logic and deployment	Helped list the platform features to cover (forms, workflows, reports)
3	D. Di Ruscio et al., 2022 [7]	Low-code development and model-driven engineering: two sides of the same coin?	Shows low-code builds on model-driven ideas: models instead of hand-written code	Supports designing the data model first and generating forms from it
4	A. C. Bock and U. Frank, 2021 [8]	Low-Code Platform	Describes typical parts of low-code platforms and their limits	Used in the limitations discussion (Section 8.4)
5	Zoho Corporation [2], [3]	Zoho Creator Help and Deluge Reference	Official guide for forms, workflows, approvals, blueprints and Deluge tasks	Main technical reference for building the app

2.5 Emerging Trends in Low-Code Development

Three trends shaped this design. First, AI-assisted building: Zoho Creator offers Zia to create forms from a sentence, which shows how fast simple apps can be started. Second, citizen development: branch staff who are not programmers can later change choices, reports and messages without a developer. Third, hyper-automation: instead of one big program, the app joins many small automations (validation, schedule, approval, blueprint) that each solve one step. The project follows this idea by keeping each Deluge script short and focused.

2.6 Proposed System

The proposed system is a Zoho Creator app called Bank Management System. It removes each limitation of Section 2.2 as follows:

- Data is entered once; transactions and loans pick the account through a lookup field.
- Deluge scripts block withdrawals that exceed the balance and calculate EMI automatically.
- An approval process routes loans above ₹5,00,000 to the manager, with a timestamp and comment.
- A daily schedule emails reminders for fixed deposits maturing in the next 7 days.
- Access is controlled through Zoho login and permission profiles.
- List, Kanban and Calendar reports and a dashboard always show live data.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator / Branch Manager	Owner of the app; runs the branch	Configure the app, manage branches, approve high-value loans, move accounts through the blueprint, view dashboard
Bank Staff (Cashier / Loan Officer)	Front-office staff	Open accounts, record deposits and withdrawals, enter loan applications, view reports
Customer	Account holder	Gives details for account opening; receives email alerts (no direct login in this version)
System (Scheduler)	Automated job	Sends daily FD maturity reminders

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall let staff open an account with mandatory customer name and agreement to bank terms.	Staff	High
FR-02	The system shall generate a unique account number automatically, starting from 100001.	System	High
FR-03	The system shall set the current balance equal to the opening deposit and reject deposits below ₹1,000.	System	High
FR-04	The system shall calculate yearly interest as FD amount × interest rate ÷ 100.	System	Medium
FR-05	The system shall send a confirmation email when an account is created.	System	Medium
FR-06	The system shall block a withdrawal or transfer that is larger than the account balance.	Staff	High
FR-07	The system shall update the account balance after every deposit, withdrawal or transfer.	System	High
FR-08	The system shall calculate monthly EMI from loan amount, interest rate and tenure.	Staff	High
FR-09	The system shall send loans above ₹5,00,000 to the branch manager for approval.	Manager	High

FR ID	Requirement	Role(s)	Priority
FR-10	The system shall email reminders daily for fixed deposits maturing within 7 days.	System	Medium
FR-11	The system shall move each new account through the stages Applied, KYC Verified, Active and Closed.	Manager	Medium
FR-12	The system shall show accounts by status (Kanban), FD maturity dates (Calendar) and total balance (dashboard).	Manager	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports with up to 1,000 records shall open within about 3 seconds on a normal connection.
NFR-02	Security	Only signed-in users of the app shall see data; users shall access only what their permission profile allows.
NFR-03	Usability	Forms shall use clear labels, drop-downs and lookups so staff can fill them without training.
NFR-04	Availability	The app shall be available 24×7 through Zoho's hosted cloud.
NFR-05	Maintainability	Rules shall be kept in short, commented Deluge scripts, one per purpose.
NFR-06	Auditability	Record changes and approvals shall be traceable through Added Time, Modified Time and approval history.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (trial plan), zoho.in data centre
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome (latest version)
API testing tool	Not used in this version
Other Zoho services	Zoho Mail (through the Deluge sendmail task)
Mobile	Not used in this version
Client hardware	Laptop with Intel Core i5, 8 GB RAM and internet connection

3.5 Use Case Model

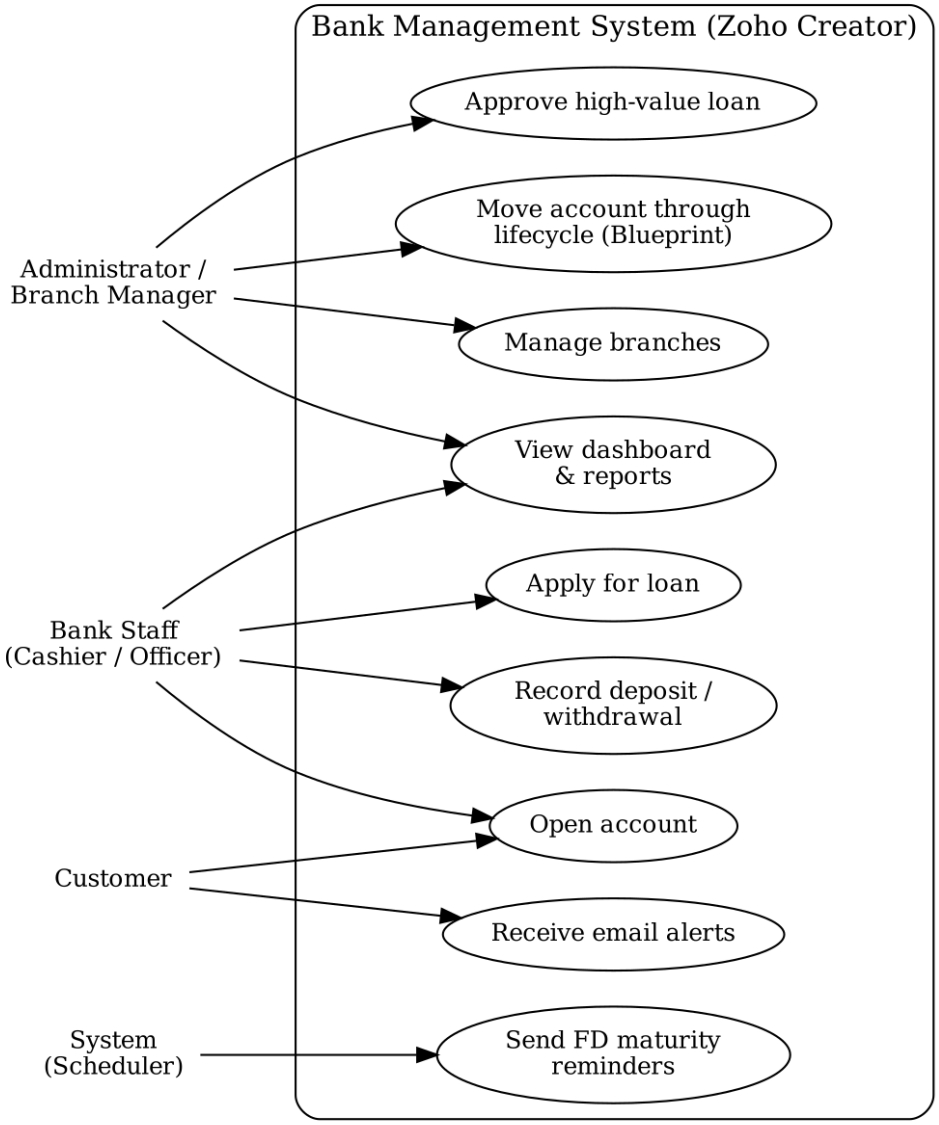


Figure 3.1: Use Case Diagram of the Bank Management System

Table 3.5: Use Case Description – Record a Withdrawal

Use Case ID	UC-01
Use Case Name	Record a Withdrawal
Actor(s)	Bank Staff (Cashier)
Pre-condition	The customer account exists and has a balance.
Main Flow	1. Cashier opens the Transaction Form. 2. Selects Withdrawal, enters amount and selects the account. 3. Clicks Submit. 4. System checks the balance, reduces it and saves the record with today's date.
Alternate Flow	3a. Amount is more than the balance: system shows "Insufficient balance" and does not save.
Post-condition	Transaction is saved and the account balance is updated.

Related FR	FR-06, FR-07

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

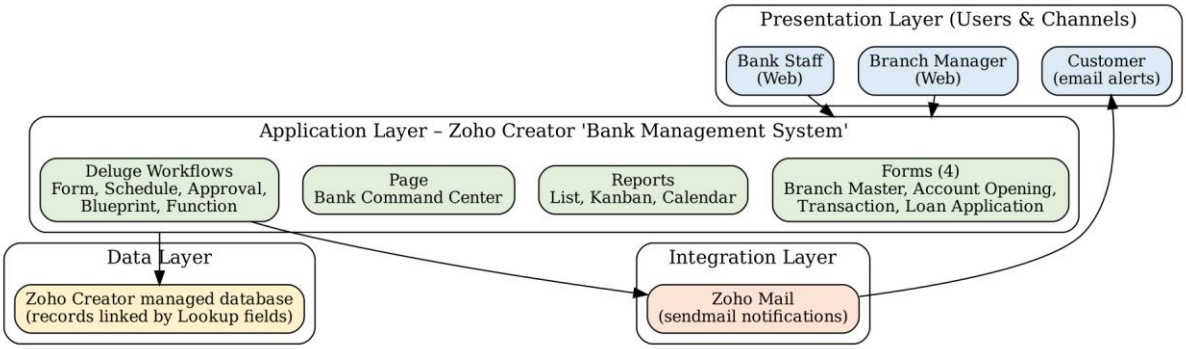


Figure 4.1: System Architecture of the Bank Management System

The architecture has four layers. The presentation layer is the Zoho Creator web app used by the branch manager and staff; customers are reached by email. The application layer holds the four forms, the reports, the Bank Command Center page and the Deluge workflows. The data layer is the database managed by Zoho Creator, where records are linked through lookup fields. The integration layer is Zoho Mail, used by the sendmail task to send confirmations and reminders.

4.2 Data Model



Figure 4.2: Entity–Relationship Diagram (Zoho Creator Schema Builder)

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Branch Master	Branch_Name, IFSC_Code, City	Account Opening Form	One-to-Many	Lookup Branch_Master in Account Opening Form
Account Opening Form	Account_Number, Customer_Name, Account_Type, Current_Balance, Account_Status	Transaction Form	One-to-Many	Lookup Account in Transaction Form
Account Opening Form	Account_Number	Loan Application Form	One-to-Many	Lookup Account in Loan Application Form
Account Opening Form	Account_Number	Nominee Details (subform)	One-to-Many	Subform Nominee_Details inside Account Opening Form

4.3 Form Design

Table 4.2: Field Specification – Account Opening Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Customer Name	Customer_Name	Name	Yes	First and last name	Account holder
Email Address	Email_Address	Email	No	Email format check	For alerts
Mobile Number	Mobile_Number	Phone	No	Country code +91	
Residential Address	Residential_Address	Address	No	Line 1/2, city, state, PIN	
PAN Number	PAN_Number	Single Line	No	Max 255 chars	KYC
Opening Deposit	Opening_Deposit	Number	No	≥ 1000 (checked by DS-01)	
Date of Birth	Date_of_Birth	Date	No	—	
Purpose of Account	Purpose_of_Account	Multi Line	No	—	
Preferred Visit Time	Preferred_Visit_Time	Time	No	—	Branch visit

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Account Type	Account_Type	Drop Down	No	Savings / Current / Fixed Deposit / Recurring Deposit	
Customer Category	Customer_Category	Radio	No	Individual / Senior Citizen / Student	
Services Required	Services_Required	Multi Select	No	ATM Card / Net Banking / SMS Alerts	
Agree To Bank Terms	Agree_To_Bank_Terms	Checkbox	Yes	Must be ticked	Consent
Customer Photo	Customer_Photo	Image	No	Single image	KYC
Branch Master	Branch_Master	Lookup	No	From Branch Master (Branch Name)	Relationship
KYC Document	KYC_Document	File Upload	No	Single file	Aadhaar/PAN copy
Current Balance	Current_Balance	Currency (INR)	No	Set by DS-01, updated by DS-03	System managed
FD Amount	FD_Amount	Currency (INR)	No	—	
Interest Rate	Interest_Rate	Percent	No	—	
Submitted On	Submitted_On	Date-Time	No	Set to current time by DS-01	System managed
Company Website	Company_Website	URL	No	URL format	Current accounts
Minimum Balance	Minimum_Balance	Decimal	No	—	
Requires Debit Card	Requires_Debit_Card	Decision Box	No	Yes / No	
Bank Terms	Bank_Terms	Rich Text	No	—	Terms text
Customer Signature	Customer_Signature	Signature	No	—	
Account Number	Account_Number	Auto Number	—	Starts at 100001	System generated
Yearly Interest	Yearly_Interest	Formula	—	FD_Amount * Interest_Rate / 100	Read only

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Account QR Code	QR_Code	QR/Barcode	—	Account No + Name	Shown in report
FD Maturity Date	FD_Maturity_Date	Date	No	—	Used by schedule and calendar
Account Status	Account_Status	Drop Down	No	Applied / KYC Pending / Approved / Active / Closed	Used by Kanban
Nominee Details	Nominee_Details	Subform	No	Columns: Name (Single Line), Relationship (Drop Down: Father, Mother, Spouse, Son, Daughter, Brother, Sister, Other)	Parent–child nominee rows

Table 4.3: Field Specification – Branch Master Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Branch Name	Branch_Name	Single Line	No	—	Shown in lookup
IFSC Code	IFSC_Code	Single Line	No	—	
City	City	Single Line	No	—	

Table 4.4: Field Specification – Transaction Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Transaction Type	Transaction_Type	Drop Down	No	Deposit / Withdrawal / Fund Transfer	
Transaction Date	Transaction_Date	Date	No	Set to today by DS-03 if empty	
Remarks	Remarks	Multi Line	No	—	
Amount	Amount	Currency (INR)	Yes	Checked against balance (DS-03)	
Account	Account	Lookup	No	From Account Opening Form (Account Number)	Relationship

Table 4.5: Field Specification – Loan Application Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Loan Type	Loan_Type	Drop Down	No	Home / Education / Personal Loan	
Tenure Months	Tenure_Months	Number	No	> 0 (checked by DS-04)	
Loan Status	Loan_Status	Drop Down	No	Applied / Approved / Rejected; default set by DS-04	
Loan Amount	Loan_Amount	Currency (INR)	No	> 0 (checked by DS-04)	> 5,00,000 goes for approval
Loan Interest Rate	Loan_Interest_Rate	Percent	No	—	Yearly rate
Monthly EMI	Monthly_EMI	Currency (INR)	No	Calculated by DS-04	System managed
Account	Account	Lookup	No	From Account Opening Form	Relationship

4.4 Process and Workflow Design

Figure 4.3 shows the most important process, recording a transaction. Other processes follow the same pattern: the loan form calculates EMI before saving, loans above ₹5,00,000 wait in the manager's Pending Tasks, and new accounts move through the blueprint stages Applied → KYC Verified → Active → Closed.

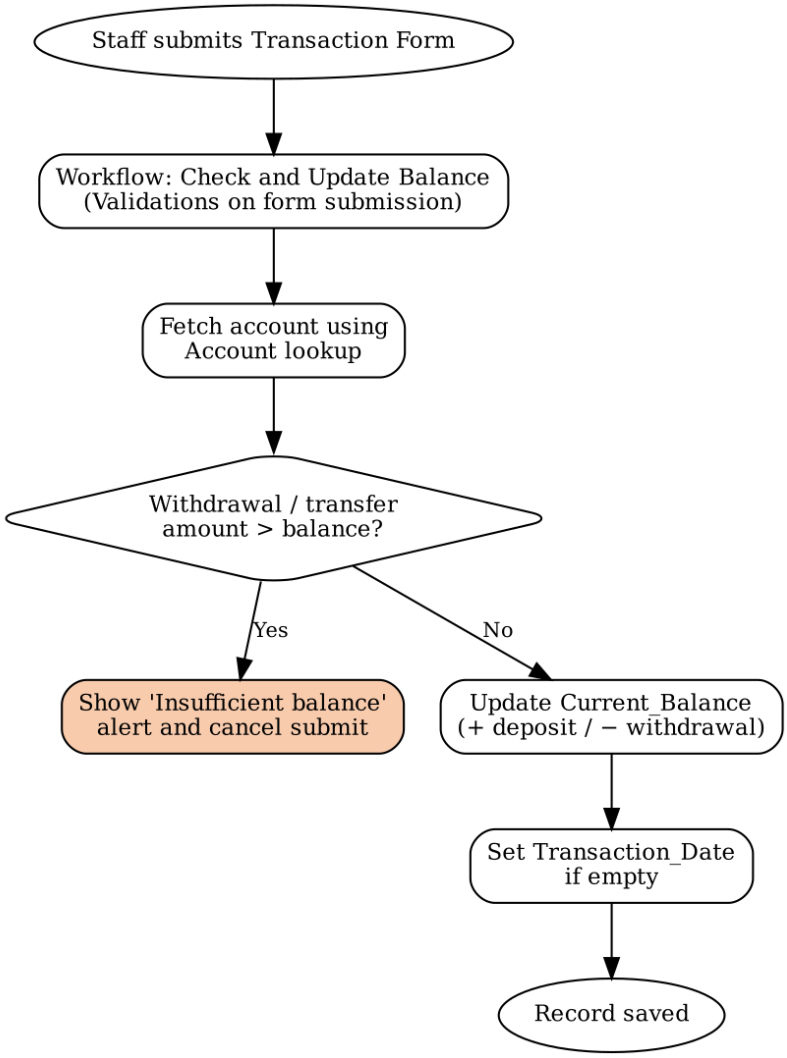


Figure 4.3: Workflow / Process Flow for Recording a Transaction

4.5 User Interface and Navigation Design

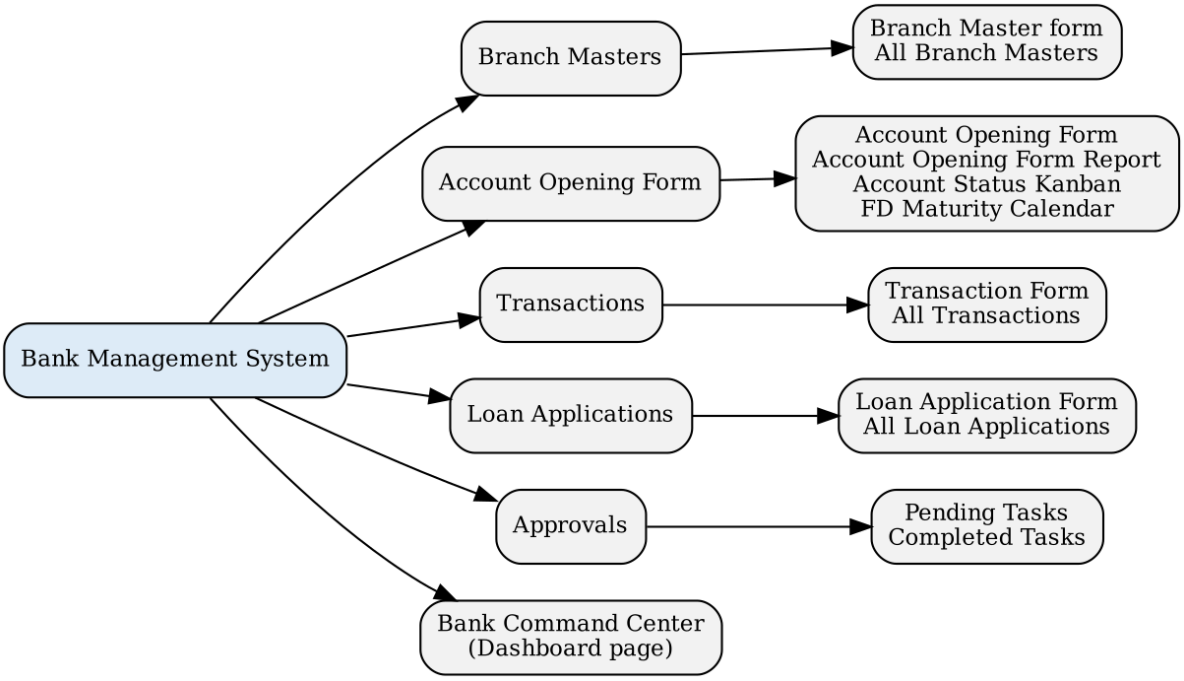


Figure 4.4: Navigation Structure of the Bank Management System

4.6 Access Control Design

Table 4.6: Role–Permission Matrix

Component	Administrator / Branch Manager	Bank Staff	Customer
Form: Branch Master	A E V D	V	—
Form: Account Opening	A E V D	A E V	— (email only)
Form: Transaction	A E V D	A V	—
Form: Loan Application	A E V D	A V	—
Approvals / Blueprint actions	Approve, Reject, Transition	—	—
Page: Bank Command Center	V	V	—

A = Add, E = Edit, V = View, D = Delete. The planned design uses the default Write profile for the administrator and a limited profile for staff. On the trial account only the owner role could be used (see Section 5.8).

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created in Zoho Creator using “Create from scratch” and named Bank Management System (link name bank-management-system). Zoho Creator then created a section in the menu for every form, with the form and its report inside. Later the Approvals section and the Bank Command Center page were added to the same menu.

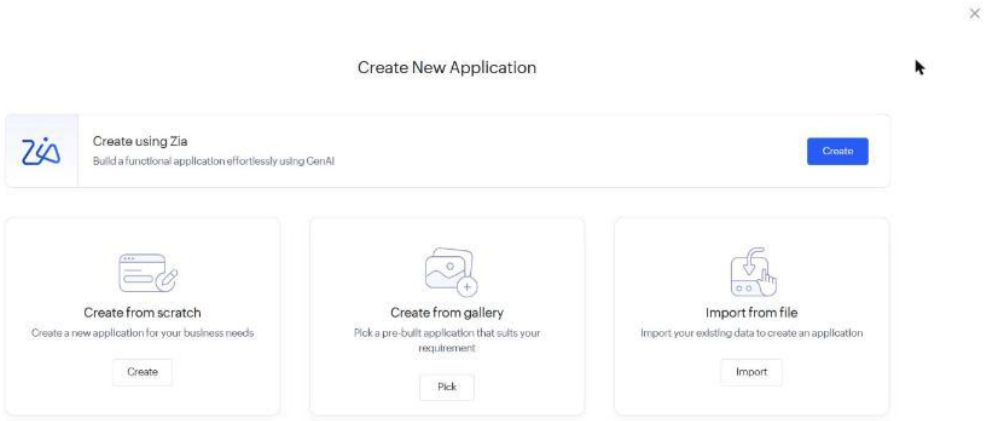


Figure 5.1: Create New Application Window in Zoho Creator

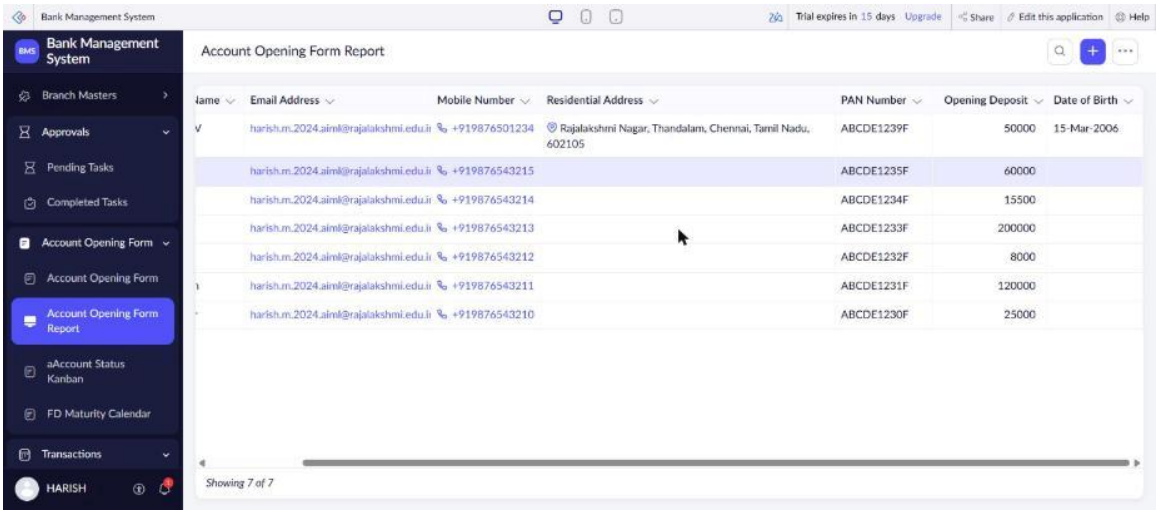


Figure 5.2: Application Home with Navigation Sections (Live Mode)

5.2 Forms and Data Capture

Four forms were built by dragging fields from the Basic and Advanced field lists in the form builder. The Account Opening form uses 27 different field types. Choices for

drop-downs, radio buttons and checkboxes were entered in the field properties. Lookups link the forms: Account Opening uses Branch Master, and both Transaction and Loan Application use Account Opening (Account Number).

Validations are done in two ways: field properties (for example Mandatory on Customer Name, Agree To Bank Terms and Amount) and Deluge scripts (minimum deposit, balance check and loan values). The Formula field calculates yearly interest as the user types; for an FD of ₹50,000 at 7% it showed ₹3,500.

A subform, Nominee Details, was added at the end of the Account Opening form. Each account can have one or more nominee rows with the nominee's Name and Relationship, added with the "Add New" link (Figure 5.8). This is a one-to-many parent–child relationship: the nominee rows are saved with the account record and shown in the record view of the Account Opening Form Report, where account 100007 shows the nominee Ramesh V (Figure 5.9).

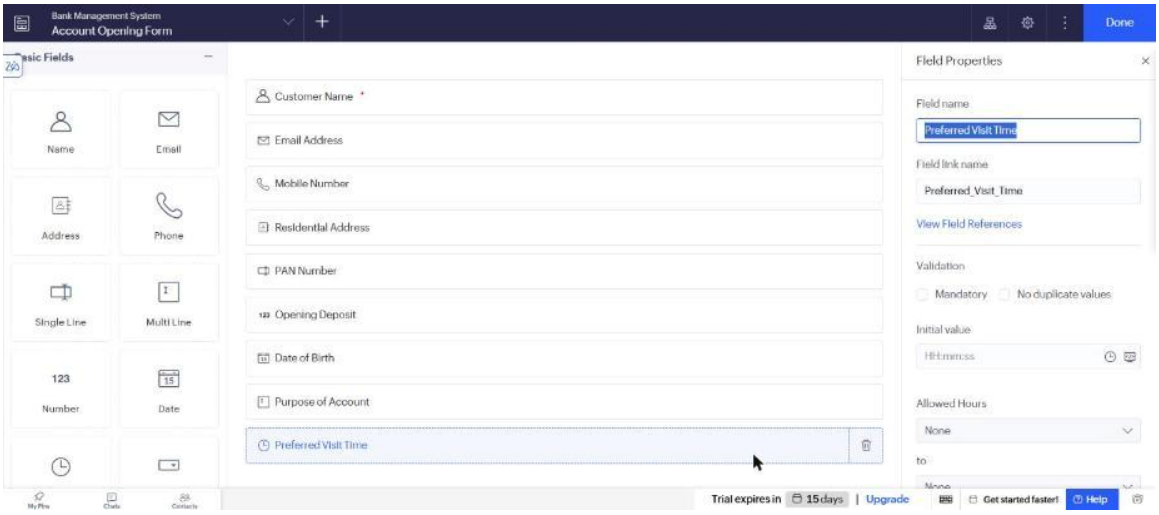


Figure 5.3: Account Opening Form in the Form Builder

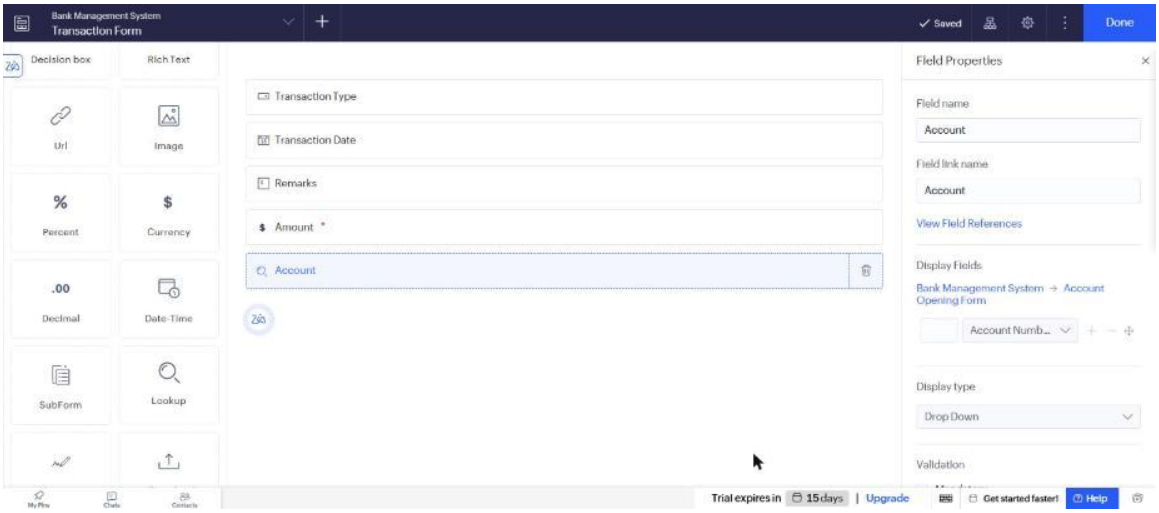


Figure 5.4: Transaction Form in the Form Builder

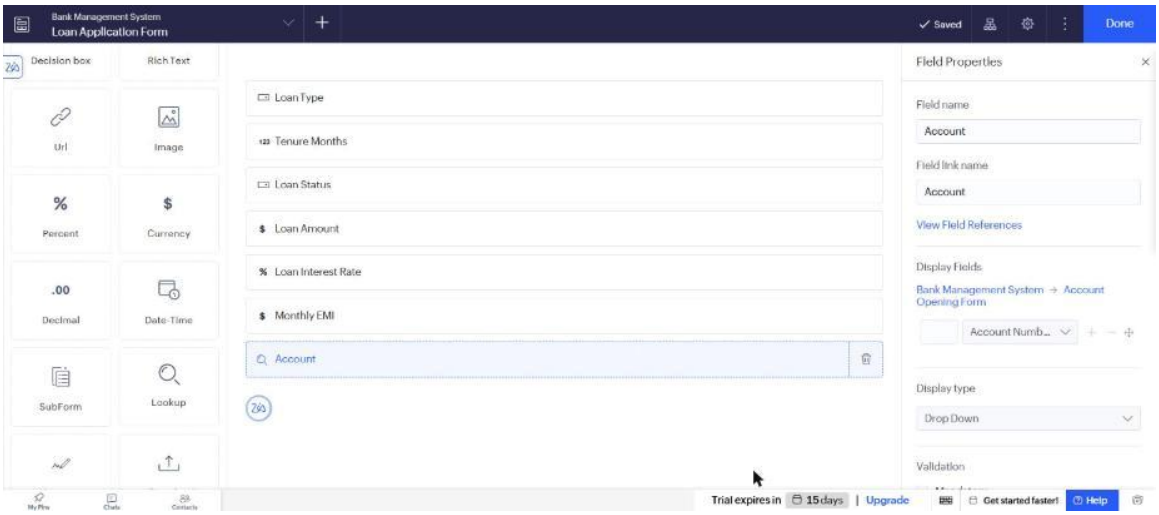


Figure 5.5: Loan Application Form in the Form Builder

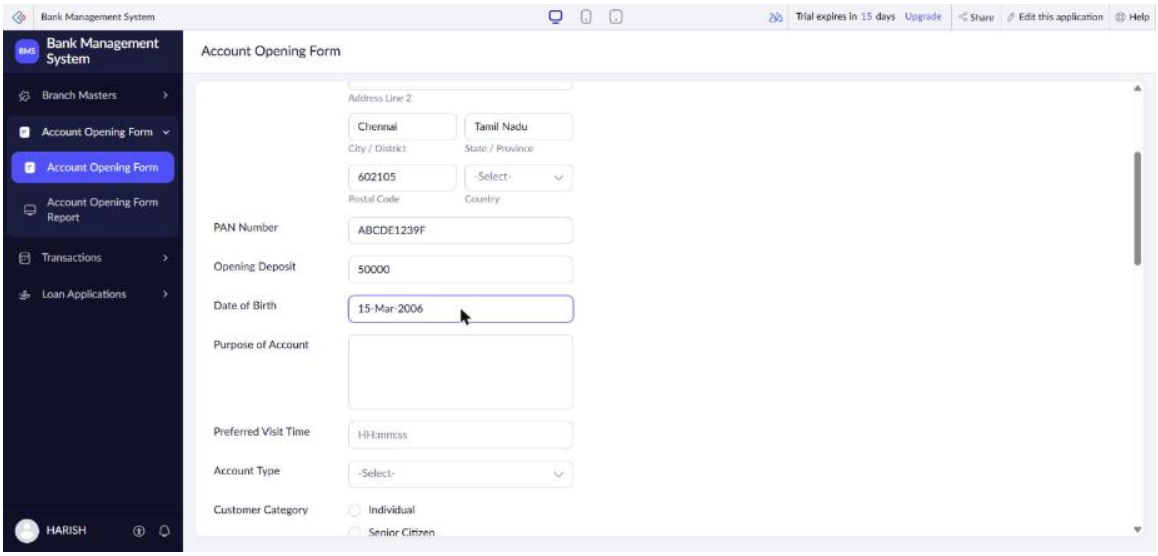


Figure 5.6: Account Opening Form in Live Mode (Address, PAN, Deposit)

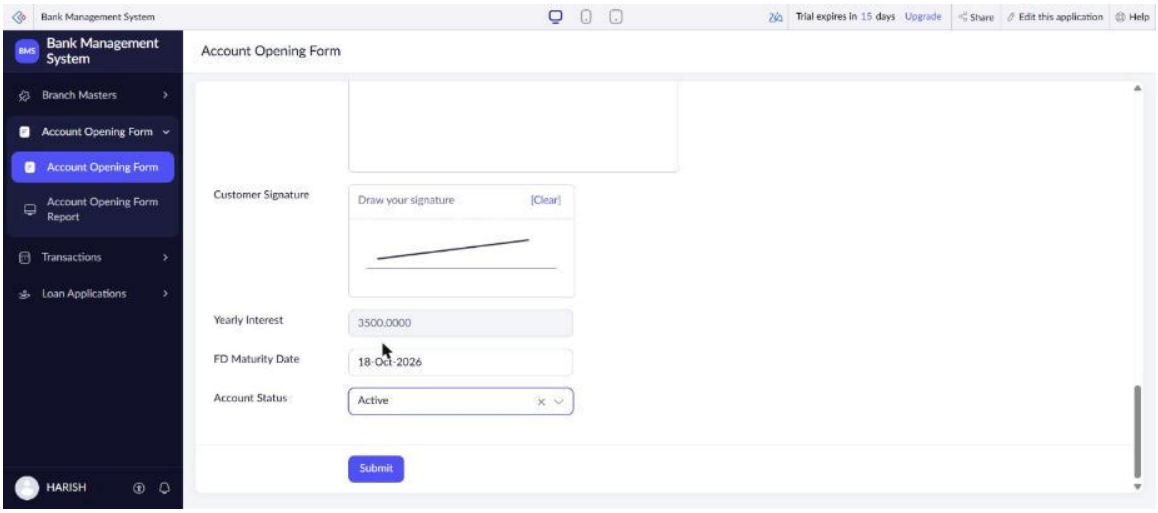


Figure 5.7: Account Opening Form in Live Mode (Signature, Formula and Status)

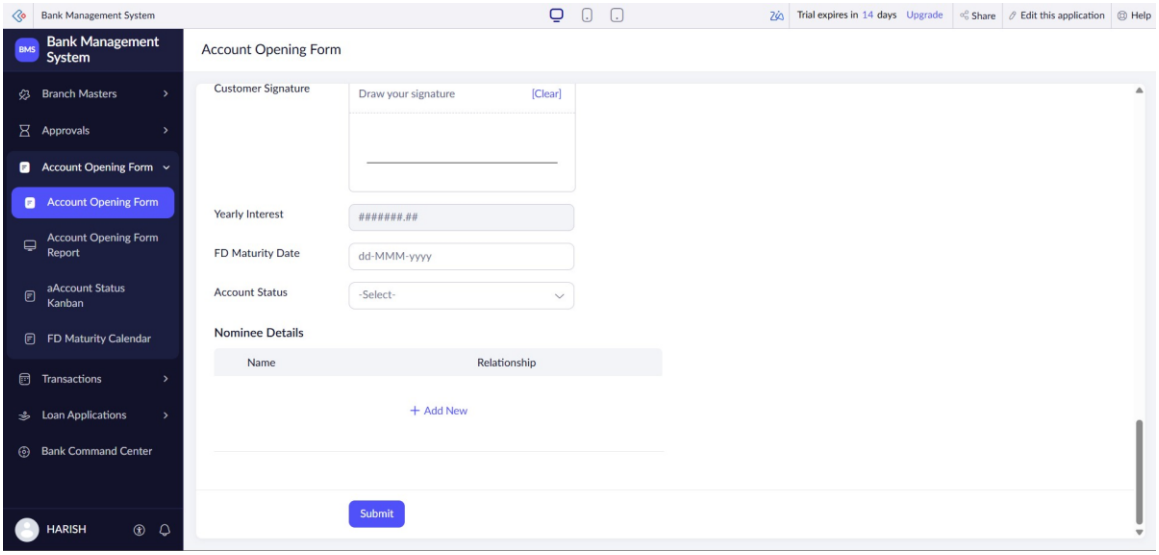


Figure 5.8: Nominee Details Subform in the Account Opening Form (Live Mode)

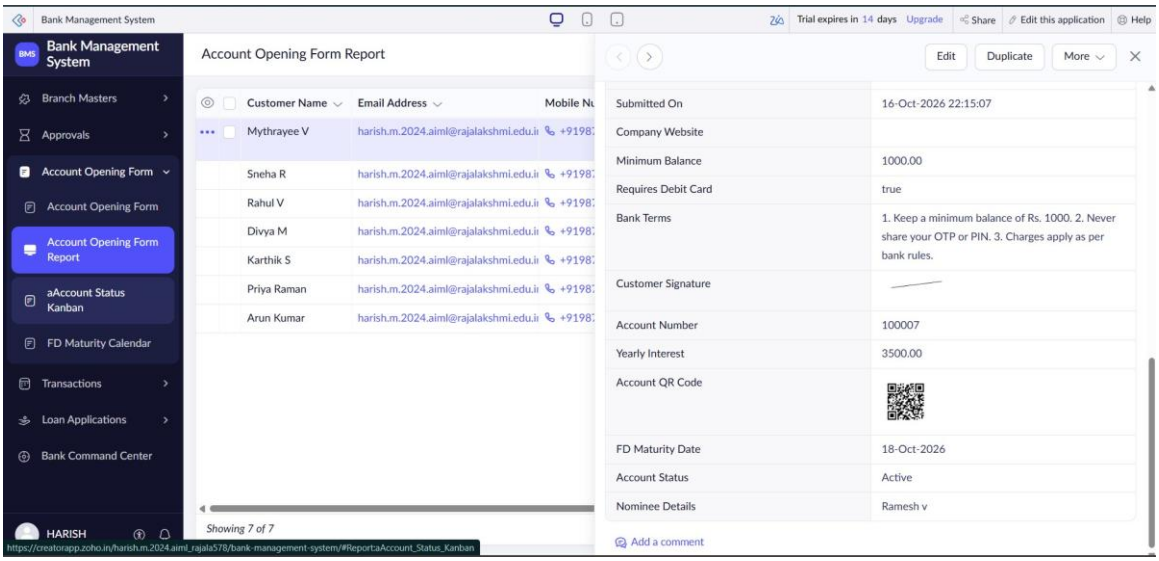


Figure 5.9: Account Record with QR Code, Signature and Nominee Details

5.3 Reports

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Account Opening Form Report	List	Account Opening Form	Default sort by Added Time; search	Add, Edit, Delete, Export	Admin, Staff
All Transactions	List	Transaction Form	Default sort; search	Add, Edit	Admin, Staff
All Loan Applications	List	Loan Application Form	Default sort; search	Add, Edit	Admin, Staff
All Branch Masters	List	Branch Master	Default sort	Add, Edit	Admin
Account Status Kanban	Kanban	Account Opening Form	Columns grouped by Account_Status	Drag card to change status	Admin
FD Maturity Calendar	Calendar	Account Opening Form	Event title Account_Number; date FD_Maturity_Date	Open record	Admin

A pivot chart was also tried. Zoho Creator asked to configure Zoho Analytics first (Figure 5.15), which is a separate service, so the chart of balances by account type was built on the dashboard page instead (Section 5.4).

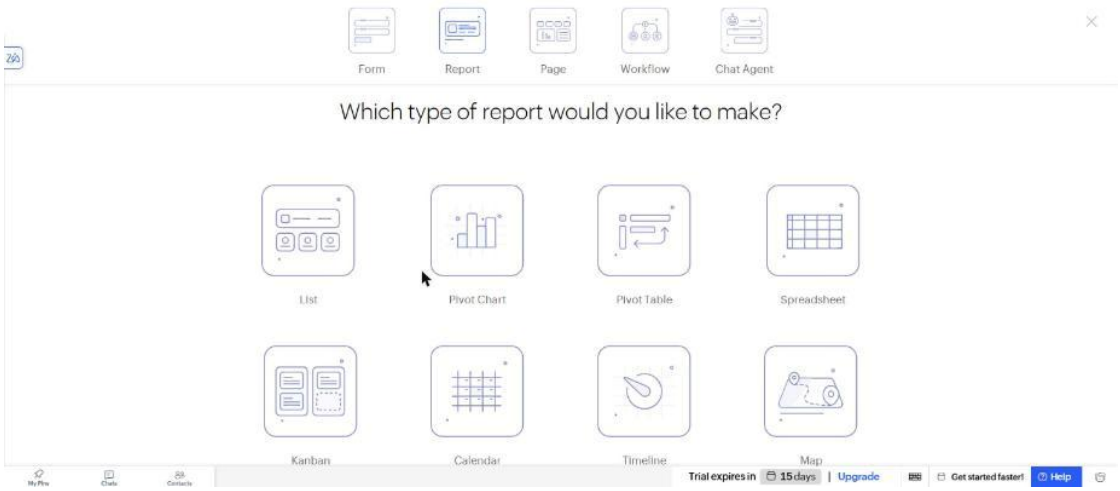


Figure 5.10: Report Types Available in Zoho Creator

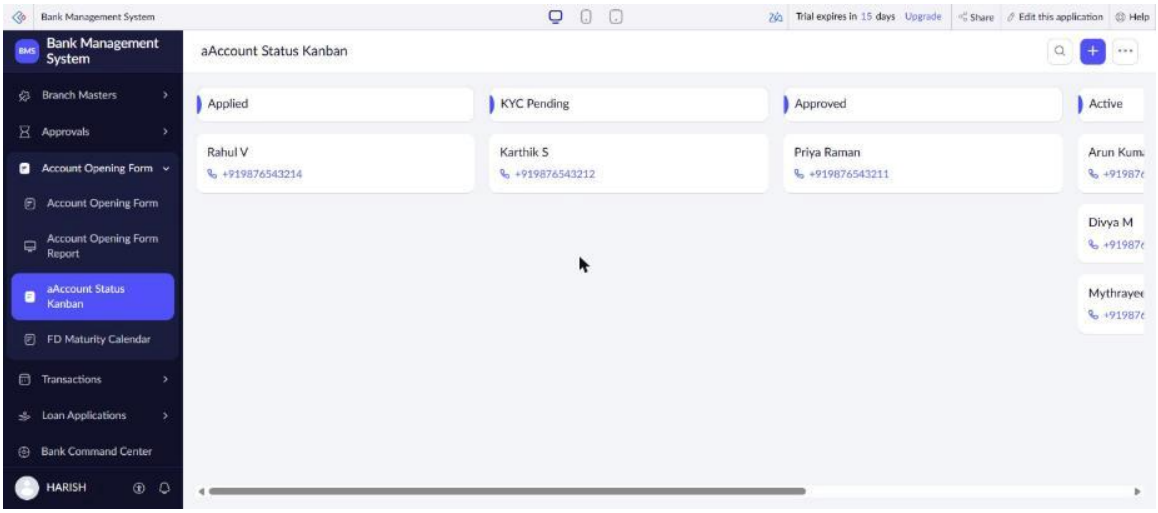


Figure 5.11: Account Status Kanban Report

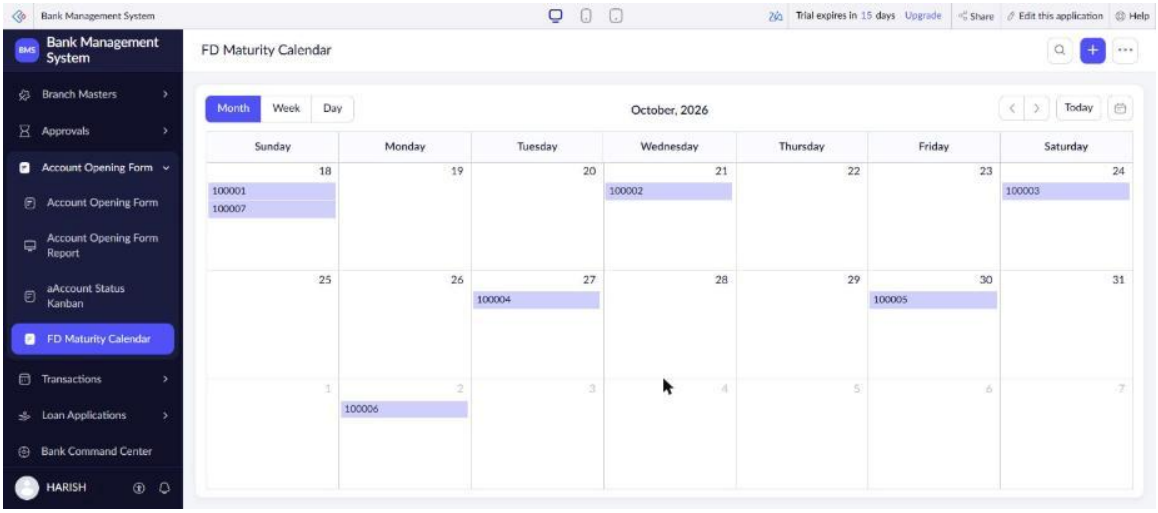


Figure 5.12: FD Maturity Calendar Report

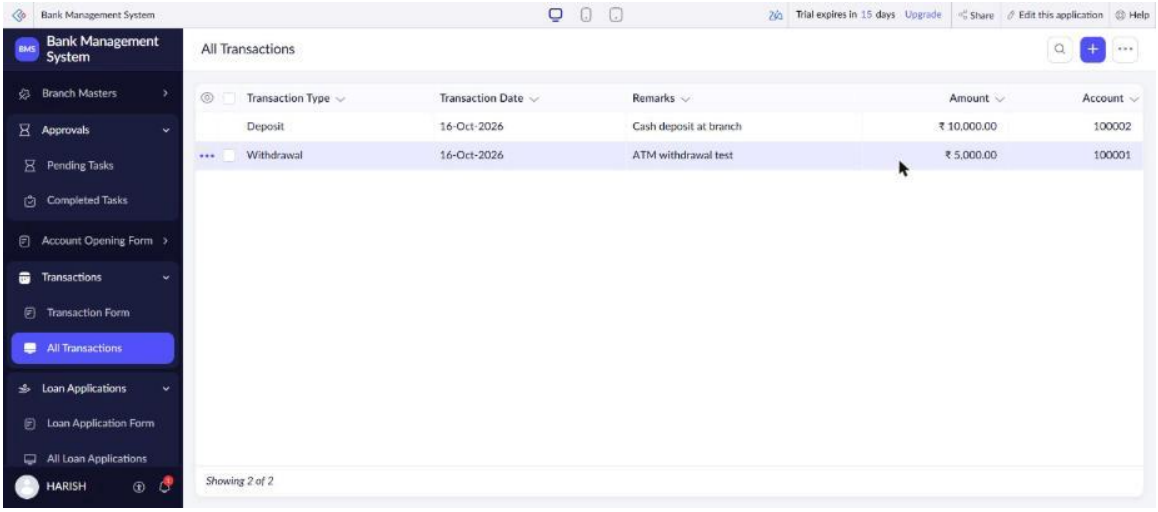


Figure 5.13: All Transactions Report

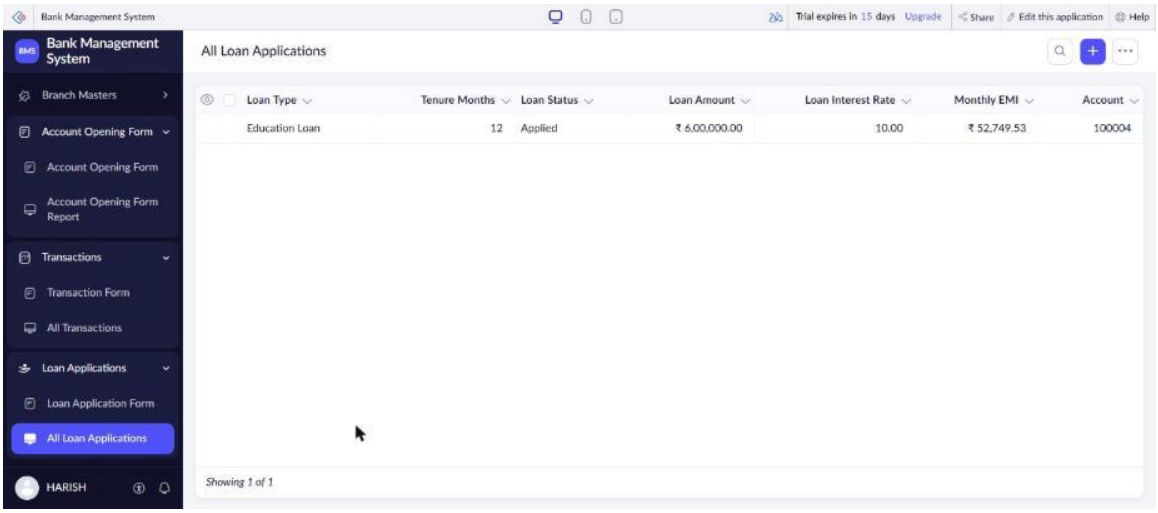


Figure 5.14: All Loan Applications Report with Calculated EMI

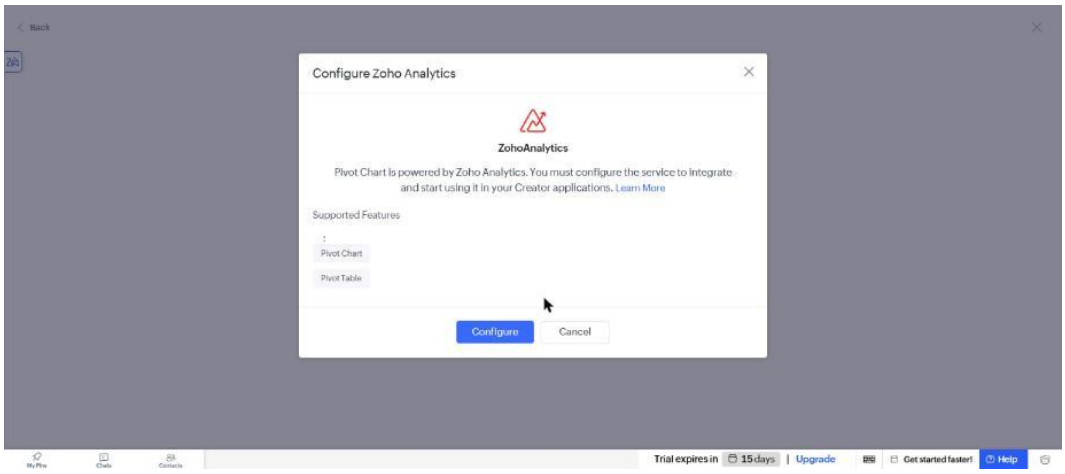


Figure 5.15: Pivot Chart Requires Zoho Analytics Configuration

5.4 Pages and Dashboards

The Bank Command Center page was built with the page builder. It has three elements: a Panel that shows the sum of Current Balance of all accounts (₹4,83,500 at the time of testing), a Column chart of total balance by Account Type, and an embedded Account Opening Form Report. The page was added to the app menu so the manager sees it in one click.



Figure 5.16: Bank Command Center Page in the Page Builder

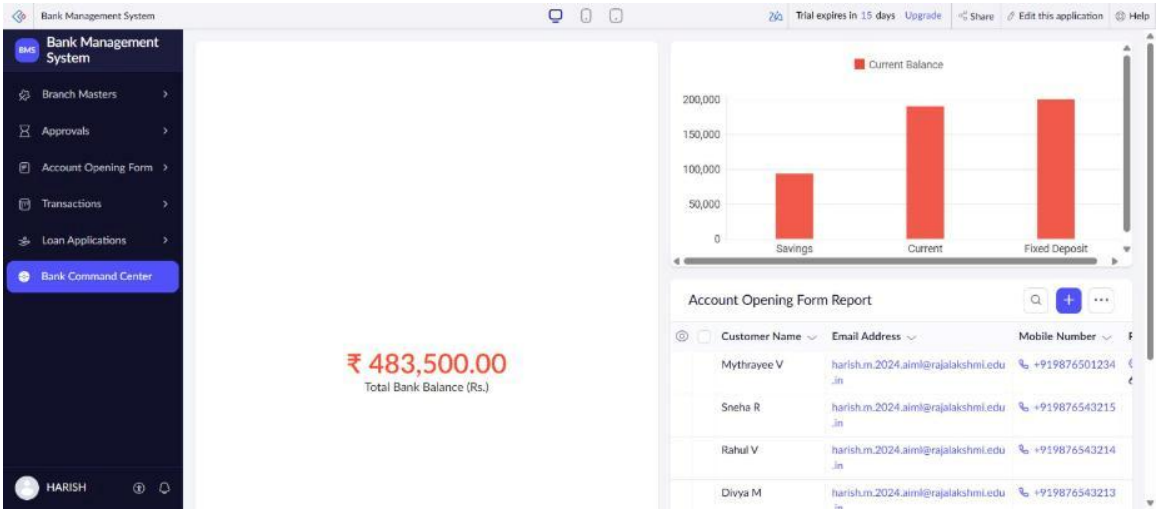


Figure 5.17: Bank Command Center Dashboard in Live Mode

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Account Opening Form → Created → Validations on form submission	Set opening balance and submission time; reject deposit below ₹1,000	input fields, zoho.currenttime, if, alert, cancel submit
DS-02	Account Opening Form → Created → Successful form submission	Send account confirmation email	sendmail task, string concatenation
DS-03	Transaction Form → Created → Validations on form submission	Check balance, update account balance, set date	fetch record by lookup, ifnull, nested if-else, update fetched record, alert

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-04	Loan Application Form → Created or Edited → Validations on form submission	Calculate monthly EMI (reducing balance) and default status	arithmetic, power(), round(), if-else, alert
DS-05	Schedule: Daily FD Maturity Reminder	Email reminders for FDs maturing in next 7 days	date functions addDay(), for each with criteria, sendmail, info
DS-06	Approval: High Value Loan Approval → Approve	Set Loan_Status to Approved	input field update
DS-07	Function: Load_Sample_Data	Load branches, link accounts, correct test data	map, keys(), insert into, getAll(), modulo, for each, info
DS-08	Batch workflow: Monthly Savings Interest Credit	Credit 3% p.a. monthly interest to Active Savings accounts	batch blocks, input record update, round()
DS-09	Function: Get_Exchange_Rates	Call a public REST API and read JSON	invokeurl, map get(), toDecimal(), info

Code 5.1: *Opening balance and minimum deposit check (DS-01)*

```
// DS-01 | Form: Account Opening | Event: Created -> Validations on form
submission
// Set the opening balance and submission time for a new account
input.Current_Balance = input.Opening_Deposit;
input.Submitted_On = zoho.currenttime;
if(input.Opening_Deposit < 1000)
{
    alert "Minimum opening deposit is Rs. 1000";
    cancel submit;
}
```

Code 5.2: *Balance check and update on every transaction (DS-03)*

```
// DS-03 | Form: Transaction | Event: Created -> Validations on form
submission
acc = Account_Opening_Form[ID == input.Account];
if(acc.count() == 0)
{
    alert "Please select a valid account.";
    cancel submit;
}
else
{
    balance = ifnull(acc.Current_Balance,0);
    if(input.Transaction_Type != "Deposit" && input.Amount > balance)
    {
        alert "Insufficient balance. Available balance: Rs. " + balance;
        cancel submit;
    }
    else
    {
        if(input.Transaction_Type == "Deposit")
```

```
        {
            acc.Current_Balance = balance + input.Amount;
        }
        else
        {
            acc.Current_Balance = balance - input.Amount;
        }
        if(input.Transaction_Date == null)
        {
            input.Transaction_Date = zoho.currentdate;
        }
    }
}
```

Code 5.3: *Loan EMI calculation (DS-04)*

```
// DS-04 | Form: Loan Application | Event: Created or Edited ->
Validations
// Calculate monthly EMI using the reducing balance formula
P = ifnull(input.Loan_Amount,0);
yearlyRate = ifnull(input.Loan_Interest_Rate,0);
N = ifnull(input.Tenure_Months,0);
if(P <= 0 || N <= 0)
{
    alert "Loan amount and tenure must be greater than zero.";
    cancel submit;
}
else
{
    R = yearlyRate / 12 / 100;
    if(R == 0)
    {
        input.Monthly_EMI = (P / N).round(2);
    }
    else
    {
        factor = (1 + R).power(N);
        input.Monthly_EMI = (P * R * factor / (factor - 1)).round(2);
    }
    if(input.Loan_Status == null)
    {
        input.Loan_Status = "Applied";
    }
}
```

DS-01 copies the opening deposit into Current Balance and stamps the submission time before the record is saved; if the deposit is below ₹1,000 it shows an alert and stops the save. DS-03 fetches the selected account through the lookup, compares the amount with the balance, and either blocks the record or changes the balance of the fetched account. In testing, a withdrawal of ₹9,99,999 from account 100001 (balance ₹25,000) was blocked with the message “Insufficient balance. Available balance: Rs. 25000.00”, while a withdrawal of ₹5,000 was saved and reduced the balance to ₹20,000.

DS-04 uses the reducing-balance formula EMI = P × R × (1+R)^N ÷ ((1+R)^N − 1), where R is the monthly rate. For ₹6,00,000 at 10% for 12 months it gave ₹52,749.53,

and for ₹7,50,000 at 8.5% for 60 months it gave ₹15,387.40. The full code of all nine scripts is given in Appendix C.

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Set Opening Balance	Form Workflow	Account Opening – Created – Validations on submission	Always	Set balance and time; block deposit < 1,000	DS-01
Account Confirmation Email	Form Workflow	Account Opening – Created – Successful submission	Always	Send confirmation email	DS-02
Check and Update Balance	Form Workflow	Transaction – Created – Validations on submission	Always	Block invalid withdrawal; update balance	DS-03
Calculate EMI	Form Workflow	Loan Application – Created or Edited – Validations	Always	Calculate EMI; default status	DS-04
Daily FD Maturity Reminder	Schedule	Daily from 17-Oct-2026	FD matures within 7 days and account Active	Send reminder emails	DS-05
High Value Loan Approval	Approval	Loan Application – record added	Loan Amount > 5,00,000	Level 1 approver: account owner; on Approve email + set status	DS-06
Account Lifecycle	Blueprint	Account Opening – record added	Always	Stages Applied → KYC Verified → Active → Closed via Verify KYC, Activate Account, Close Account	—

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Monthly Savings Interest Credit	Batch Workflow	Account Opening – On Schedule, monthly from 01-Nov-2026	Savings and Active (in script)	Add monthly interest to Current Balance, 1000 records per block	DS-08

A batch workflow, Monthly Savings Interest Credit, runs on the 1st of every month and processes accounts in blocks of 1,000 records. For each Active Savings account it adds one month of interest at 3% per year to the Current Balance (DS-08, Figure 5.22). A report workflow was not needed, because the edit trigger of DS-04 (“Created or Edited”) recalculates EMI whenever a loan is edited.

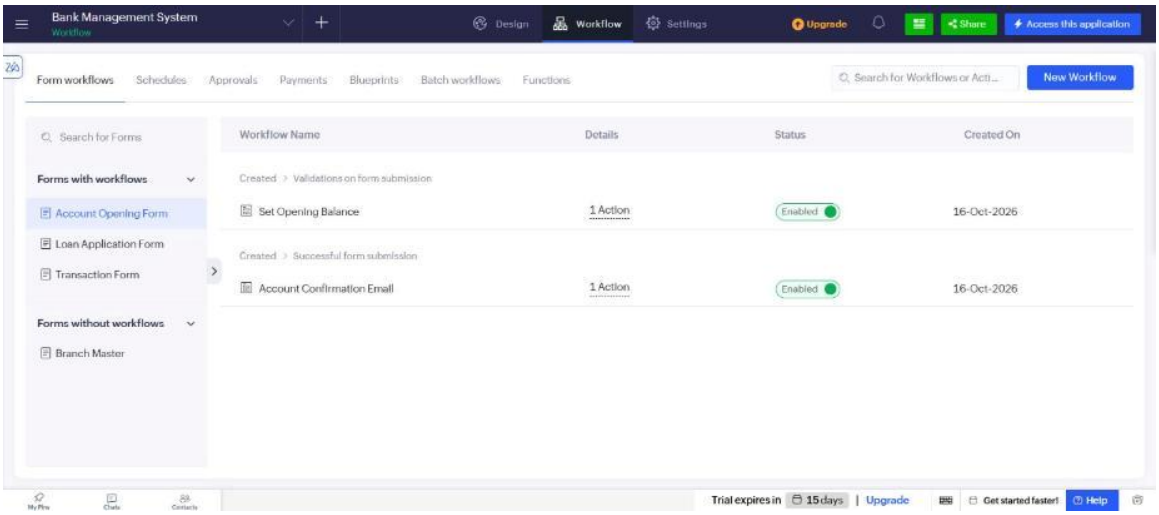


Figure 5.18: Form Workflows Configured for the Account Opening Form



Figure 5.19: Deluge Editor with the Check and Update Balance Script

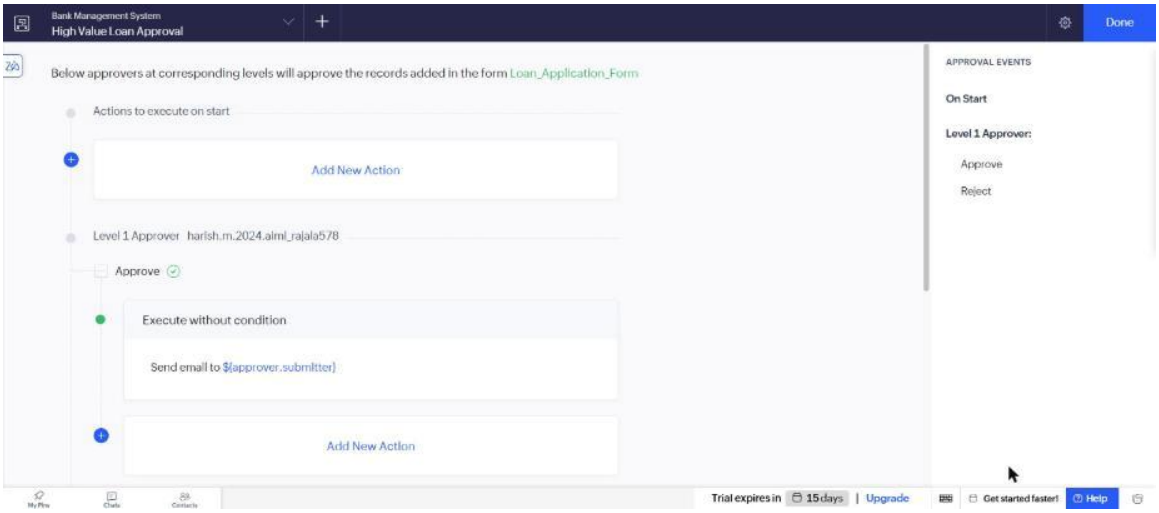


Figure 5.20: High Value Loan Approval Configuration

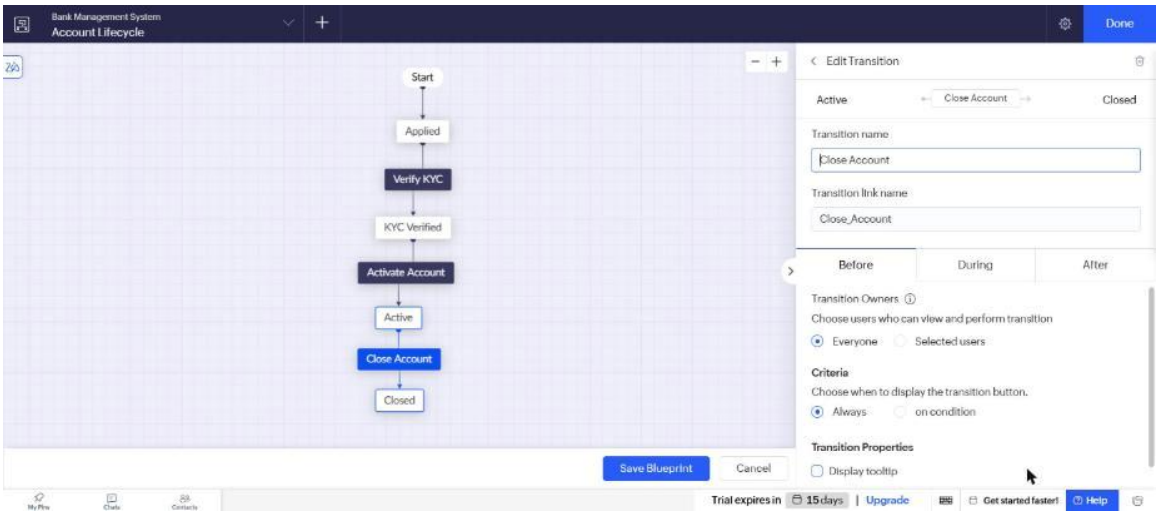


Figure 5.21: Account Lifecycle Blueprint with Stages and Transitions

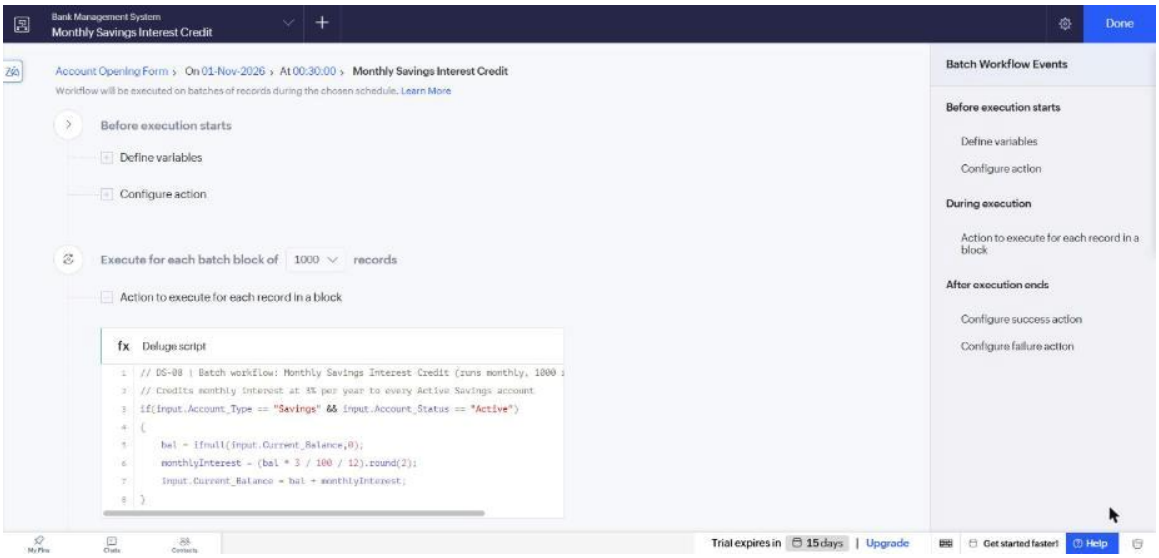


Figure 5.22: Batch Workflow for Monthly Savings Interest Credit

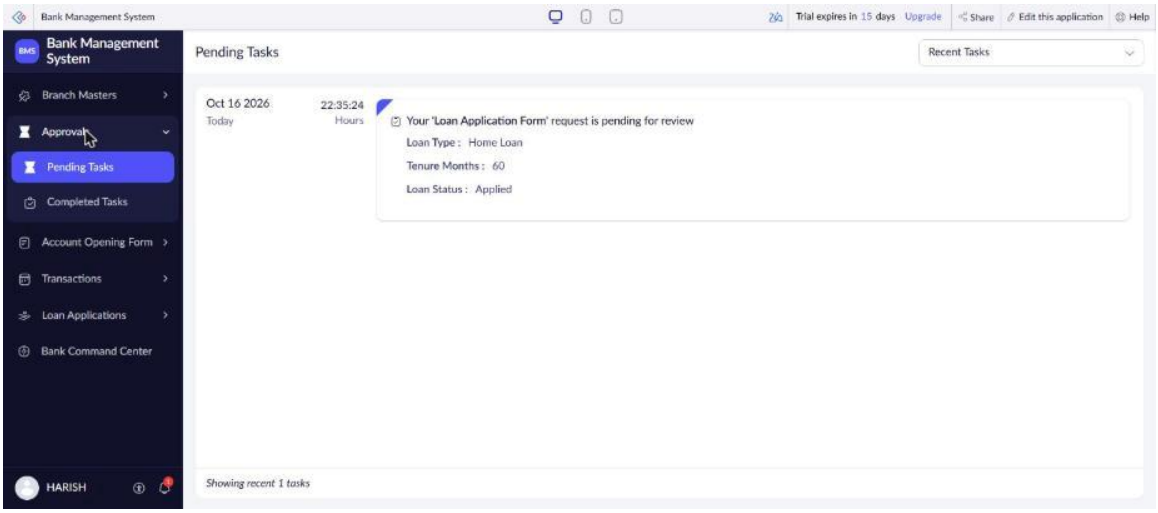


Figure 5.23: Loan of ₹7,50,000 Waiting in Pending Tasks

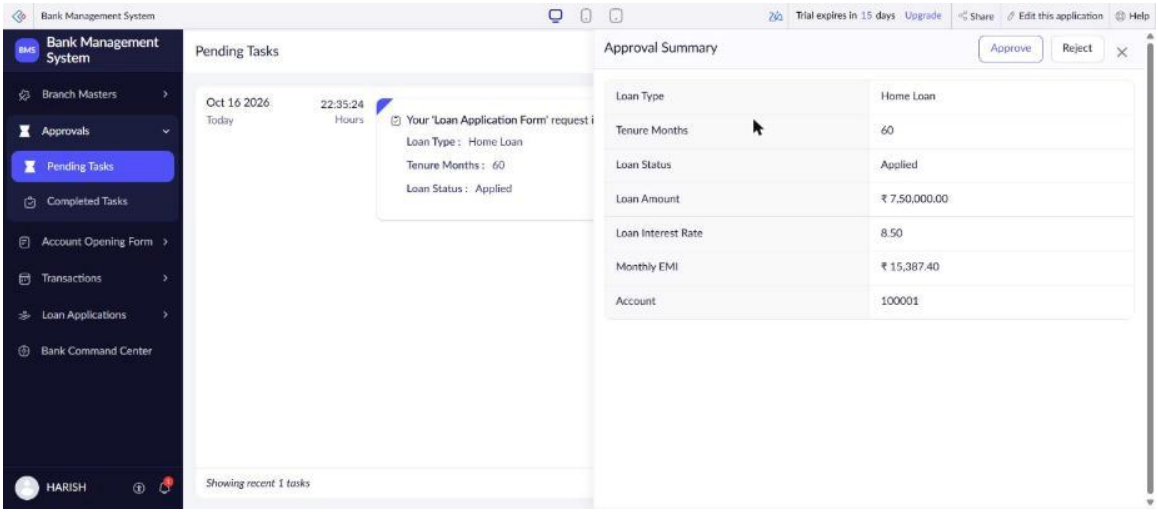


Figure 5.24: Approval Summary with Approve and Reject Buttons

5.7 Integrations

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Email notifications	Zoho Mail via Deluge sendmail	sendmail task (DS-02, DS-05) and approval email action	Zoho account (built-in)	Account confirmation, FD maturity reminder, approval notification
External REST API	Frankfurter exchange-rate API (api.frankfurter.app)	Deluge invokeurl, GET, JSON response (DS-09)	None (public API)	Today's value of 1 INR in USD and EUR for the remittance desk

Two integrations were built. Zoho Mail is used through the sendmail task. An external REST API is called from the function Get_Exchange_Rates with invokeurl: it sends a GET request to the public Frankfurter API and reads the JSON reply. When run on 16-

Oct-2026 it returned 1 INR = 0.01053 USD and 0.00936 EUR, so ₹1,00,000 = 1,053.00 USD (Figure 5.26). Testing the Creator REST API in Postman and webhooks were not done, because they need OAuth client credentials that were not created for this trial account.

Code 5.4: *Notification script using Zoho Mail (DS-05, daily schedule)*

```
// DS-05 | Schedule: Daily FD Maturity Reminder (daily)
todayDate = zoho.currentdate;
endDate = todayDate.addDay(7);
count = 0;
for each acc in Account_Opening_Form[FD_Maturity_Date >= todayDate &&
FD_Maturity_Date <= endDate && Account_Status == "Active"]
{
    sendmail
    [
        from :zoho.adminuserid
        to :zoho.adminuserid
        subject :"FD Maturity Reminder - Account " + acc.Account_Number
        message :"Dear Customer,<br><br>Your Fixed Deposit of Rs. " +
acc.FD_Amount + " in account " + acc.Account_Number + " will mature on "
+ acc.FD_Maturity_Date + ".<br>Please visit the branch to renew or
withdraw.<br><br>Regards,<br>Bank Team"
    ]
    count = count + 1;
}
info "FD reminders sent: " + count;
```

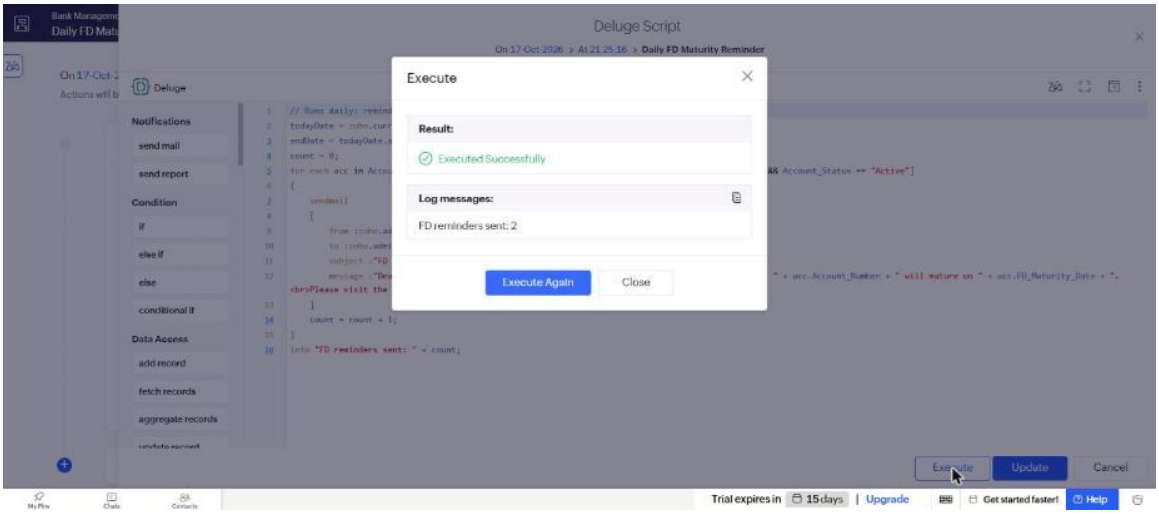


Figure 5.25: Schedule Script Executed: “FD reminders sent: 2”

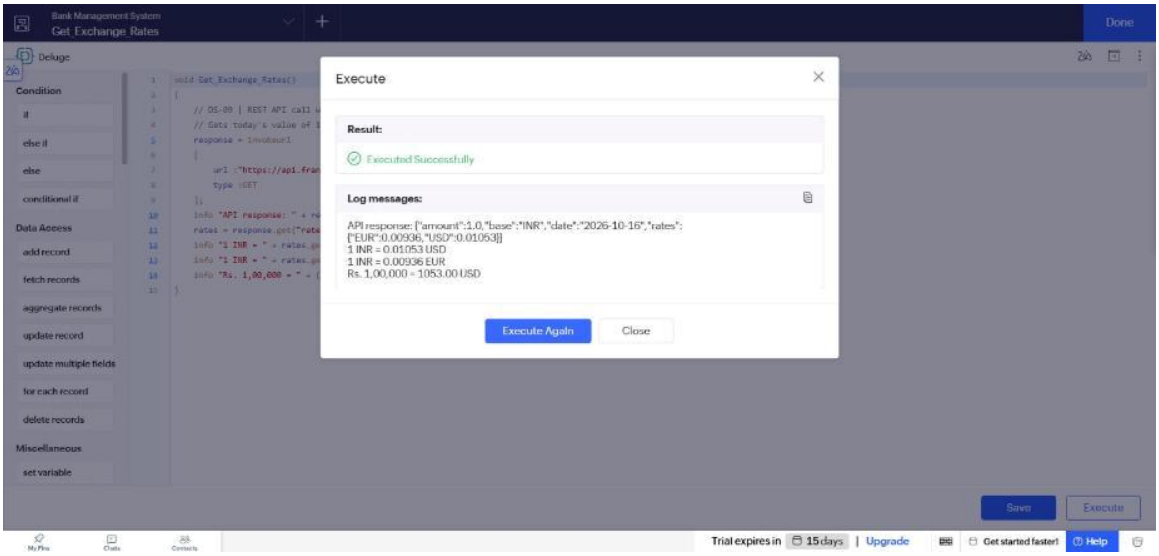


Figure 5.26: REST API Call with invokeurl – Execution Log

5.8 User Management and Role-Based Access

Settings → User Permissions shows two default permission profiles, Read and Write (Figure 5.27); the app is shared with each user as User, Developer or App admin with one of these profiles. Record-level access was set up in the Customer portal profile (Figure 5.28): customers can add and view records in the Account Opening, Loan Application and Transaction forms, but "View all" is turned off, so each customer sees only their own records. Customers cannot edit or delete records, and they have no access to the Bank Command Center page. Custom internal roles (Branch Manager, Cashier) could not be created: on this trial account the Add Permission and Roles buttons do not open, and the browser console shows a Zoho script error ("isShareRestricted is not defined").

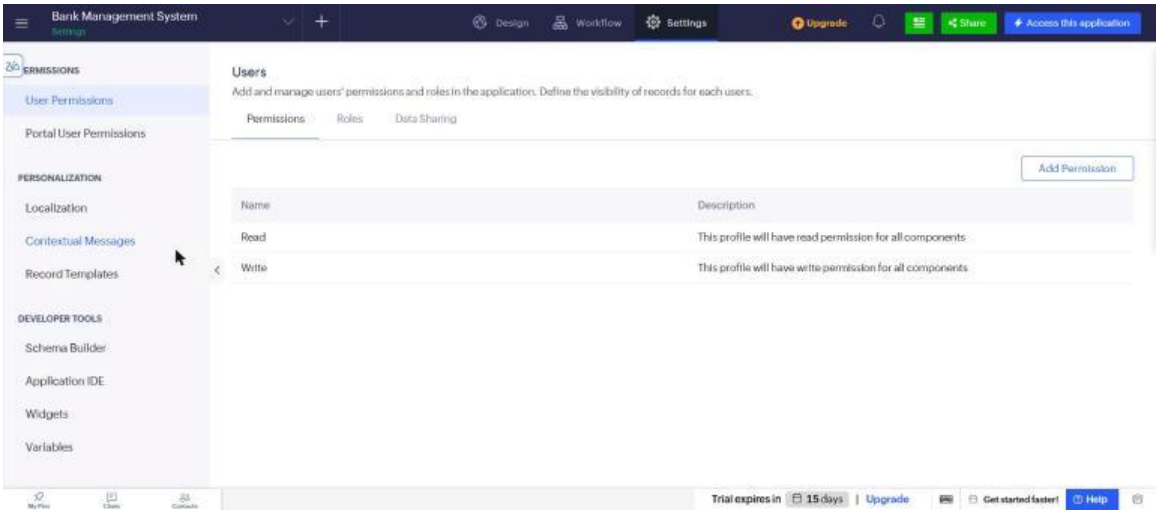


Figure 5.27: User Permissions – Read and Write Profiles

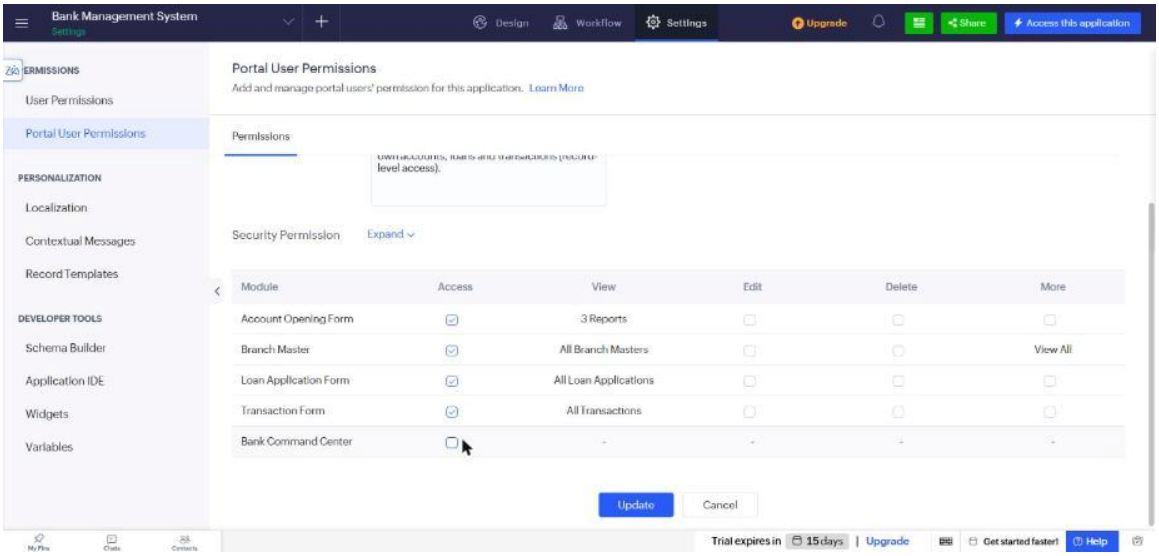


Figure 5.28: Customer Portal Profile with Record-Level Access

5.9 AI and Analytics Features

Analytics is provided by the dashboard page: a panel that adds up all balances and a column chart of balance by account type (Figure 5.17). Pivot reports need Zoho Analytics to be configured (Figure 5.15), and Zia features were not used, so AI-based prediction (for example loan default risk) is listed as future work.

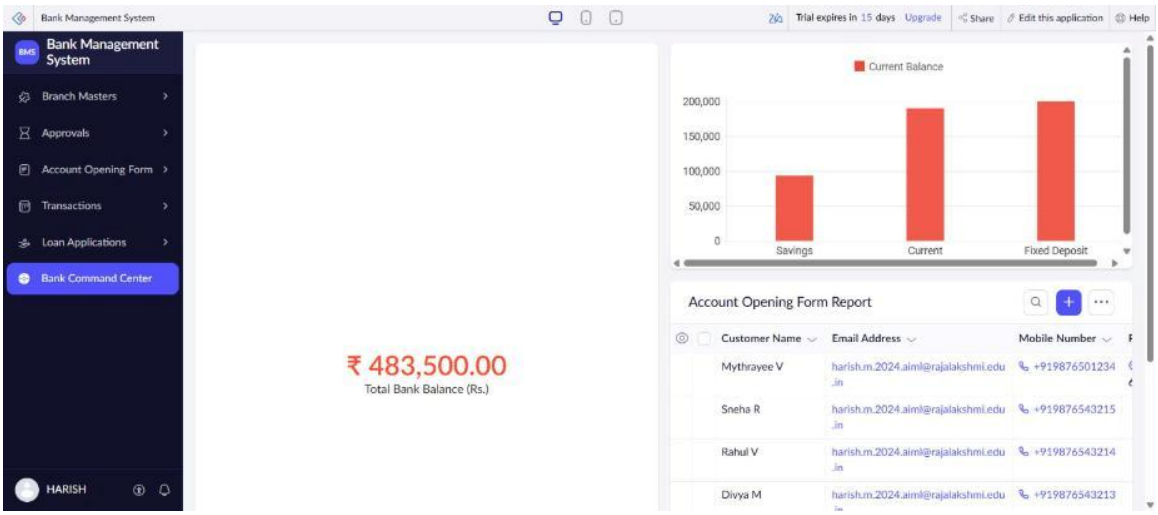


Figure 5.29: Analytics on the Dashboard: Total Balance and Balance by Account Type

5.10 Pro-Code Extension

A pro-code widget was written in HTML, CSS and JavaScript: a Loan EMI Calculator that shows the monthly EMI, total interest and the first six months of the repayment schedule. It uses the Zoho Creator Widget SDK (ZOHO.CREATOR.API.getAllRecords) to read the All Loan Applications report and

show live data from the app. It was packaged as emi_widget.zip (plugin-manifest.json + app/widget.html, full code in Appendix C.10), uploaded in Settings → Widgets as an internally hosted widget with index file /widget.html (Figure 5.30), and placed on the Bank Command Center page with the page builder. In live mode (Figure 5.31) it calculates an EMI of Rs. 52,749.53 for Rs. 6,00,000 at 10% for 12 months, matching the Deluge script DS-04, and shows "Live from app: 2 loan applications, total Rs. 1350000.00", which matches the Loan Application records.

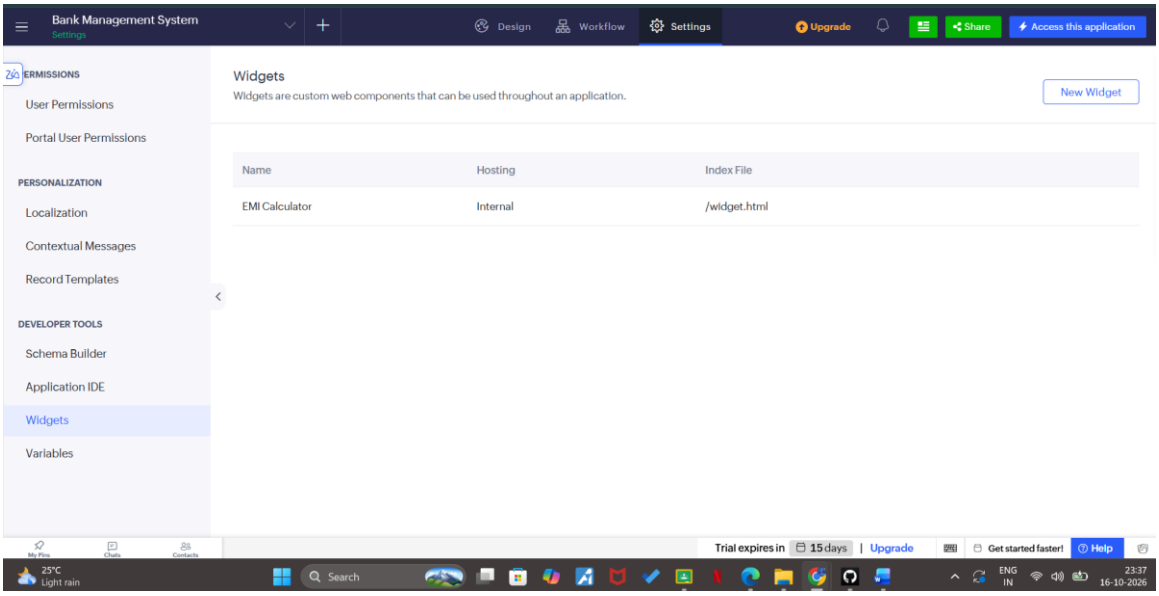


Figure 5.30: Pro-code Widget Uploaded in Settings → Widgets

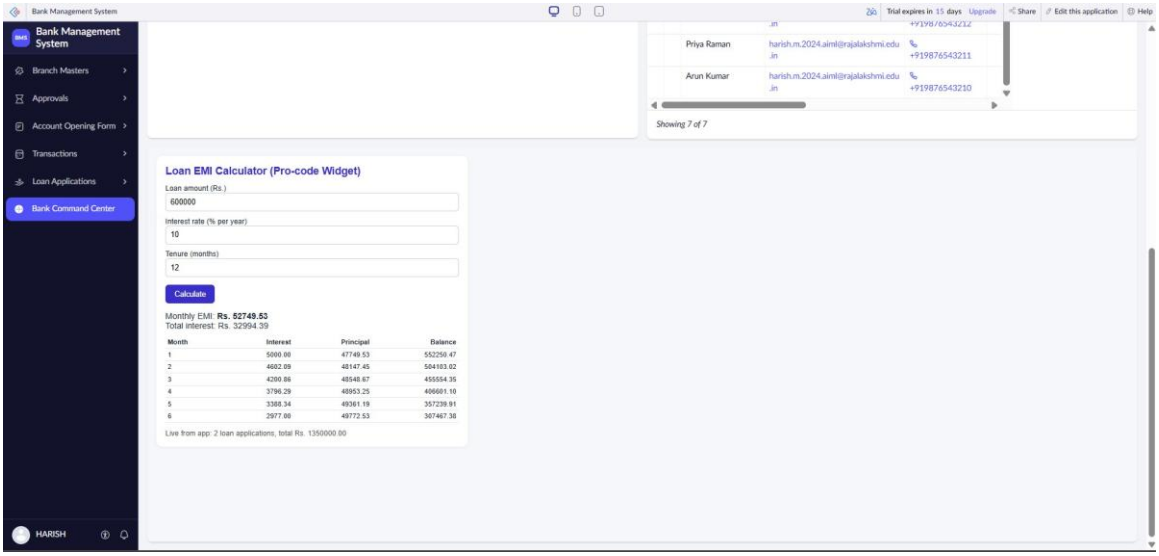


Figure 5.31: EMI Calculator Widget on the Bank Command Center Page (Live Mode)

Zoho Catalyst is Zoho's serverless platform for pro-code back-end logic. The planned Catalyst extension is a Node.js Basic I/O function, credit_score(pan), that would call a credit bureau API and return a risk score. The Calculate EMI workflow would call it

with invokeurl in the same way Get_Exchange_Rates calls the Frankfurter API (Section 5.7), and store the score on the loan record. Catalyst needs its own project in the Zoho account, so it was not created for this trial build; Deluge custom functions (Load_Sample_Data, Get_Exchange_Rates) were used as the server-side logic instead.

5.11 Mobile and Portal Access

Zoho Creator builds phone and tablet layouts of every form automatically. The phone layout of the forms was checked in the Design tab (Figure 5.32), and the same app opens in the Zoho Creator mobile app without changes. A Customer portal profile with record-level access was configured (Section 5.8); publishing the portal and inviting real customers was not done in this version.

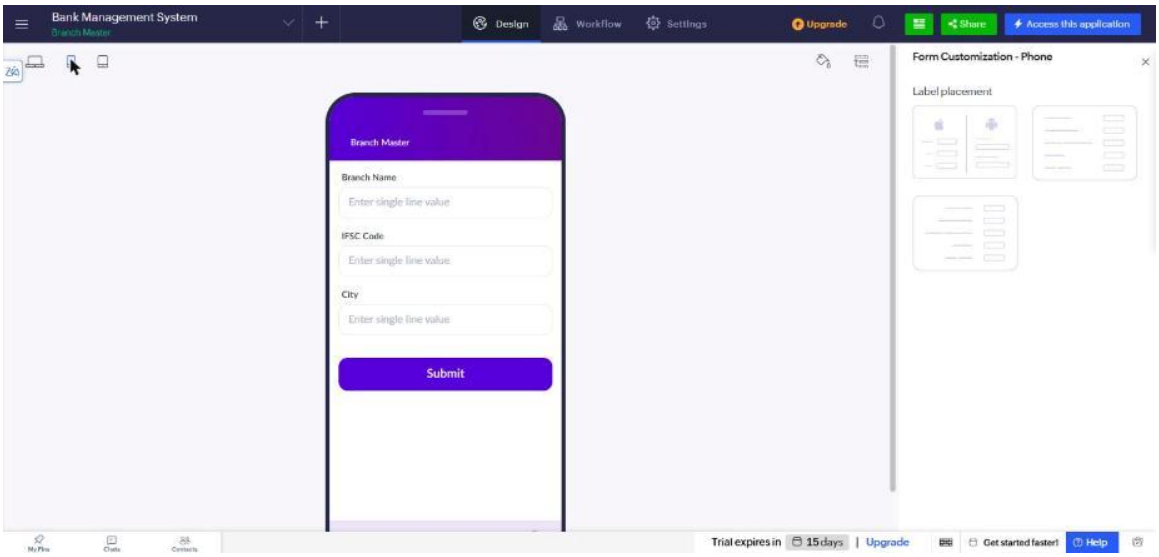


Figure 5.32: Phone Layout of a Form in the Design Tab

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was done in three levels. Unit testing: each Deluge script was saved in the editor (which checks syntax) and scripts with info statements were run with Execute. Integration testing: records were submitted on the live forms to check that lookups, workflows and the formula worked together. Scenario testing: a full flow was run – open an account, make a withdrawal and a deposit, apply for two loans and approve one – and the reports and dashboard were checked for correct totals.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Submit account form with name and terms ticked	Mythrayee V, Savings, deposit 50,000	Record saved	Record saved and form cleared	Pass
TC-02	FR-02	Auto account number	New account	Next number after 100006	Account 100007 created	Pass
TC-03	FR-03	Opening balance set	Deposit 50,000	Current Balance = 50,000	Included in dashboard total	Pass
TC-04	FR-04	Yearly interest formula	FD 50,000, rate 7%	3,500	3500.0000 shown	Pass
TC-05	FR-05	Confirmation email	Submit new account	Email sent	Email received for account 100007 (Fig 6.2)	Pass
TC-06	FR-06	Withdrawal more than balance	9,99,999 from 100001 (balance 25,000)	Alert, record not saved	Alert “Insufficient balance… 25000.00”	Pass
TC-07	FR-07	Valid withdrawal	5,000 from 100001	Saved; balance 20,000	Saved; total balance matched	Pass
TC-08	FR-07	Deposit	10,000 to 100002	Balance +10,000	Saved (type corrected, BUG-04)	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-09	FR-07	Transaction date default	Date left empty	Today's date set	16-Oct-2026 set	Pass
TC-10	FR-08	EMI calculation	6,00,000, 10%, 12 months	52,749.53	₹52,749.53	Pass
TC-11	FR-08	Default loan status	Status left empty	Applied	Applied	Pass
TC-12	FR-09	High-value loan approval	Home loan 7,50,000	Goes to Pending Tasks	Shown in Pending Tasks	Pass
TC-13	FR-09	Approve loan	Approve with comment	Status Approved	Loan_Status = Approved	Pass
TC-14	FR-10	FD reminder schedule	Execute script	Reminders for FDs in 7 days	"FD reminders sent: 2"; email received (Fig 6.3)	Pass
TC-15	FR-11	Blueprint on new account	Sneha R record: Verify KYC, then Activate Account	Stages change in order; next button shown	Applied → KYC Verified → Active (Fig 6.4)	Pass
TC-16	FR-12	Kanban and Calendar	Open reports	Grouped by status; FD dates shown	As expected	Pass
TC-17	FR-12	Dashboard total	Open page	Sum of balances	₹4,83,500 (matches records)	Pass
TC-18	FR-01	Add nominee in subform	Ramesh V, Father	Nominee saved with account	Shown in record of 100007	Pass

6.3 Debugging Deluge Scripts

Errors were found in three ways. The Deluge editor shows syntax and field errors with the line number when a script is saved (for example "You cannot assign MAP value to Name field" at line 21). The Execute button runs a script and shows info messages in the log, which was used to count reminders and accounts (Figure 6.1). Alert messages in validation scripts were used to see the values that a rule was checking.

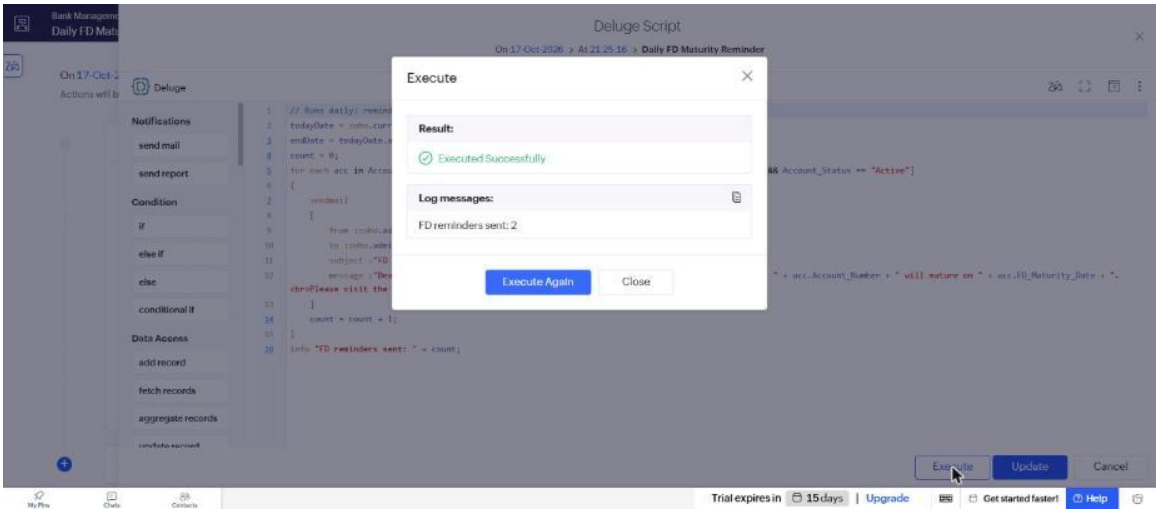


Figure 6.1: Deluge Execute Output with Info Log Message

Email workflows were checked in the inbox of the signed-in user. The Account Confirmation Email workflow (DS-02) sent "Account Opening Request Received – 100007" with the account number, type and opening deposit (Figure 6.2). The Daily FD Maturity Reminder schedule (DS-05) sent "FD Maturity Reminder – Account 100001" for the FD of Rs. 25,000 maturing on 18-Oct-2026 (Figure 6.3).

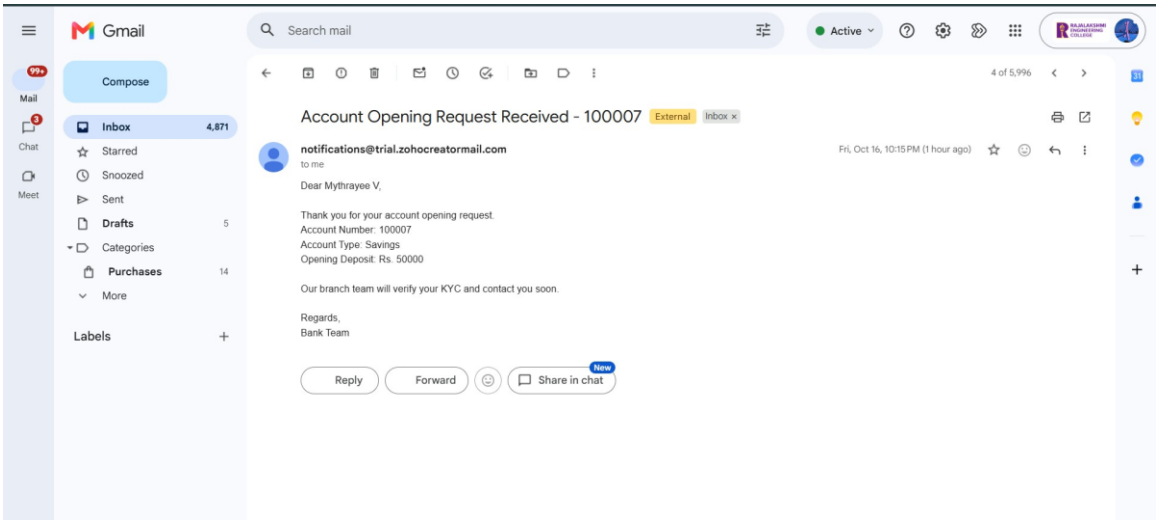


Figure 6.2: Account Confirmation Email Received in the Inbox (DS-02)

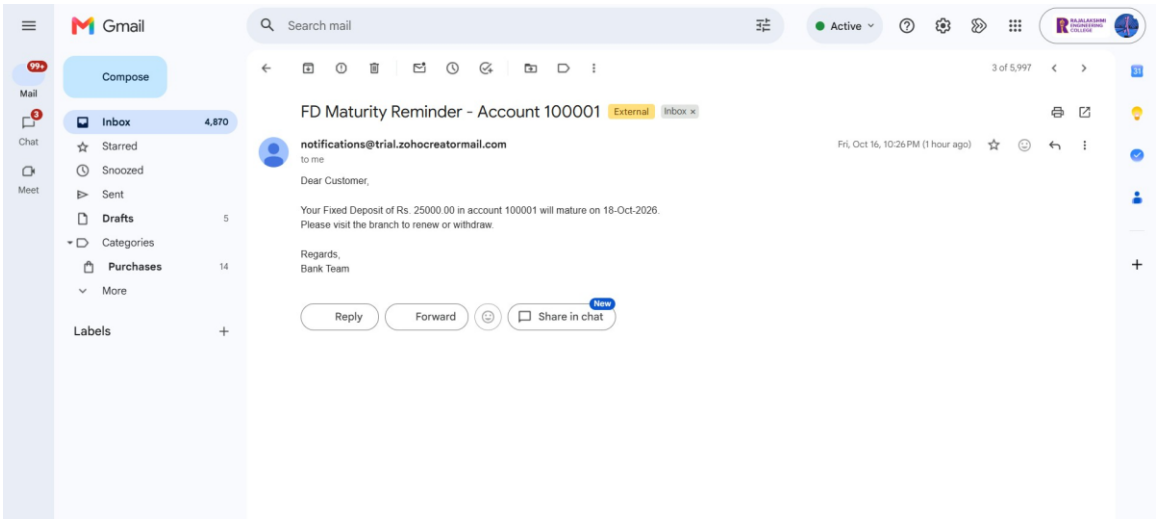


Figure 6.3: FD Maturity Reminder Email Received in the Inbox (DS-05)

The Account Lifecycle blueprint (TC-15) was tested live on the record of Sneha R. The record started in the stage Applied with the button Verify KYC; after Verify KYC it moved to KYC Verified with the button Activate Account; after Activate Account it moved to Active, where only Close Account is offered (Figure 6.4). This confirms that staff can only move an account through the stages in the defined order.

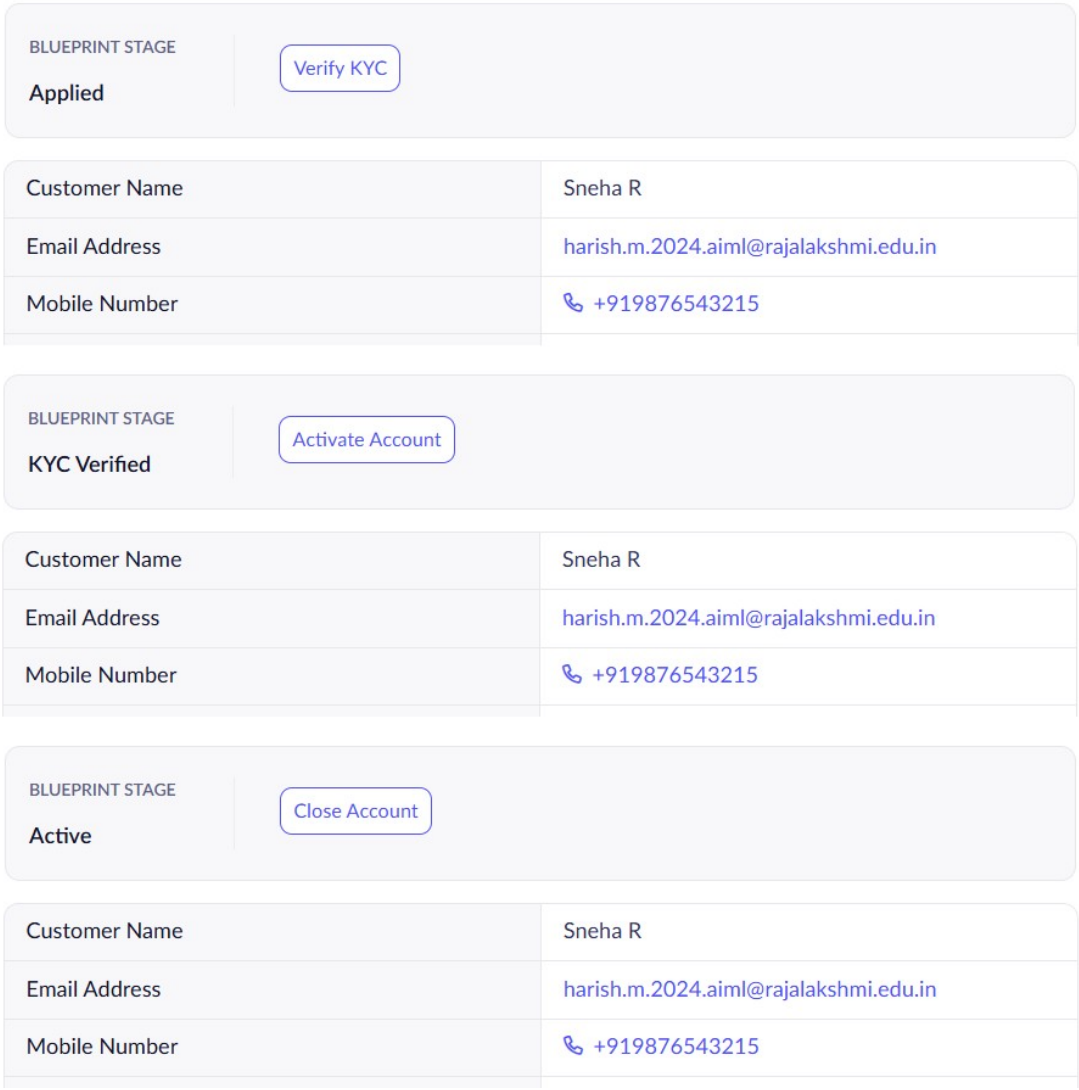


Figure 6.4: Blueprint Run: Applied → KYC Verified → Active

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Function save error: "You cannot assign MAP value to Name field"	A Name field cannot be set with a map in insert	Set Customer_Name.first_name and .last_name separately	Closed
BUG-02	Insert failed: Agree To Bank Terms should not be blank	Mandatory checkbox missing in insert	Added Agree_To_Bank_Terms value	Closed
BUG-03	Sample accounts had no branch	First run failed after inserting branches, so the whole run was rolled back	Rewrote function to create branches only if missing and link accounts	Closed

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-04	A deposit was saved as Withdrawal during testing	Drop-down kept the earlier choice	Corrected type and balance with the function; check type before submit	Closed
BUG-05	Drop-down showed Choice 1, Choice 2, Choice 3	Default choices not replaced	Removed defaults and added real choices	Closed
BUG-06	Pivot chart could not be created	Needs Zoho Analytics setup	Used page chart instead	Open (by design)

6.5 User Acceptance Testing

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
[Name of user 1]	[e.g., Bank Staff]	[Feedback to be collected]	[ ]
[Name of user 2]	[e.g., Branch Manager]	[Feedback to be collected]	[ ]

User acceptance testing with real users has not been done yet. These rows must be filled with feedback from at least two users before submission.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator moves an app from development to production through Environments: changes are made in Development, published to Stage for testing, and then published to Production for real users. This feature is not part of the trial plan, so the app was built and used directly in the Production (live) environment. The deployment followed these steps:

- Build and save each form, report, page and workflow in the builder (Development work).
- Test every script with Execute and with sample records created by Load_Sample_Data (Stage-style testing).
- Run the test cases in Table 6.1 on the live app and fix the defects in Table 6.4.
- Check the live URL and the phone layout, then share the app with users (User / Developer / App admin) with a permission profile.
- On a paid plan: enable Environments, make changes in Development, Publish to Stage, test, then Publish to Production.

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed and passed or explained	✓
2	Scripts saved without errors	✓
3	Sample data loaded and checked in reports	✓
4	Approval and schedule tested with Execute / live data	✓
5	Roles and permissions configured for each user type	✗ (only owner role on trial)
6	Environments used (Dev → Stage → Prod)	✗ (not available on trial)
7	Test data removed before go-live	[ ]

7.2 Performance Optimisation

Response times of the live app were measured from the browser on 16-Oct-2026 (5 requests each). The Bank Command Center page took 478 ms on the first (cold) request and about 210 ms after that; the Account Opening Form Report averaged about 219 ms (range 160–274 ms). Both are well inside the 3-second target of NFR-01. Table 7.2 lists the design changes that keep scripts fast.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Fetch the account once and reuse it	DS-03	Fetch per check	One fetch per transaction	Fewer database calls
Criteria in fetch instead of looping all records	DS-05	Loop all accounts	Only matching FDs fetched	Faster schedule
Validation scripts instead of on-success updates	DS-01, DS-03, DS-04	Save then fix	Values set before save	One save per record

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Zoho login (only invited users can open the app)	Whole application	Unauthorised access
Permission profiles (Read / Write)	Settings → User Permissions	Unwanted edits or deletes
Mandatory fields and validation scripts	All forms, DS-01, DS-03, DS-04	Wrong or incomplete data
Approval for high-value loans	Loan Application Form	Unchecked large loans
HTTPS hosting by Zoho	All traffic	Data theft in transit

7.4 Governance and Administration

Administration is done from the Settings page (Figure 7.1), which holds user permissions, the schema builder and developer tools. Every record keeps Added User, Added Time and Modified Time, and the approval process keeps the approver's comment and time in Completed Tasks, which gives a basic audit trail.

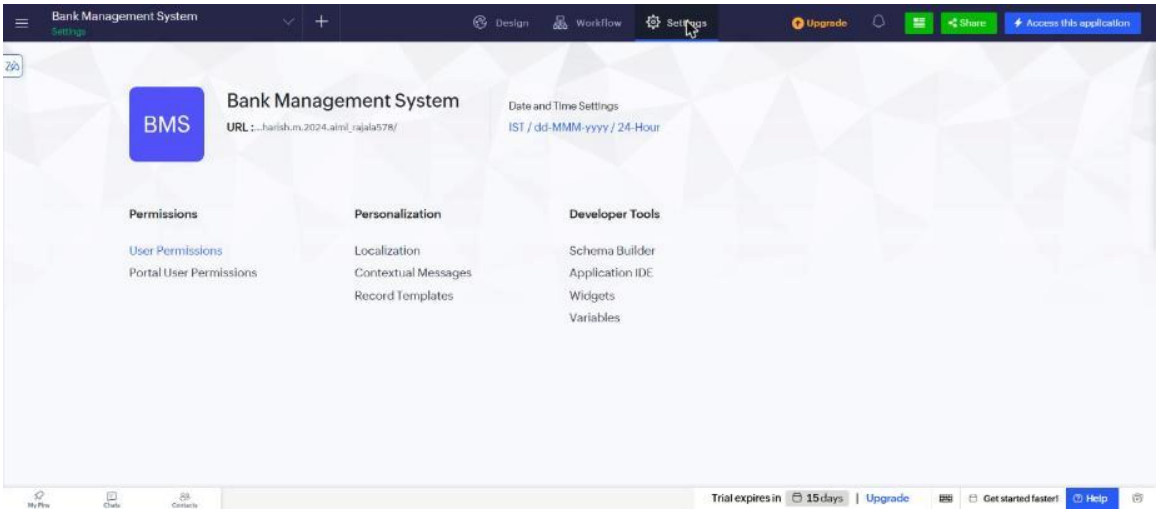


Figure 7.1: Application Settings (Administration)

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The scenario below shows the app end to end. A new customer account was opened on the live form (Figure 8.1). The cashier tried to withdraw more than the balance and the app stopped it (Figure 8.2). A home loan of ₹7,50,000 was entered; it went to the manager's Pending Tasks and, after approval, its status changed to Approved with the EMI already calculated (Figure 8.3).

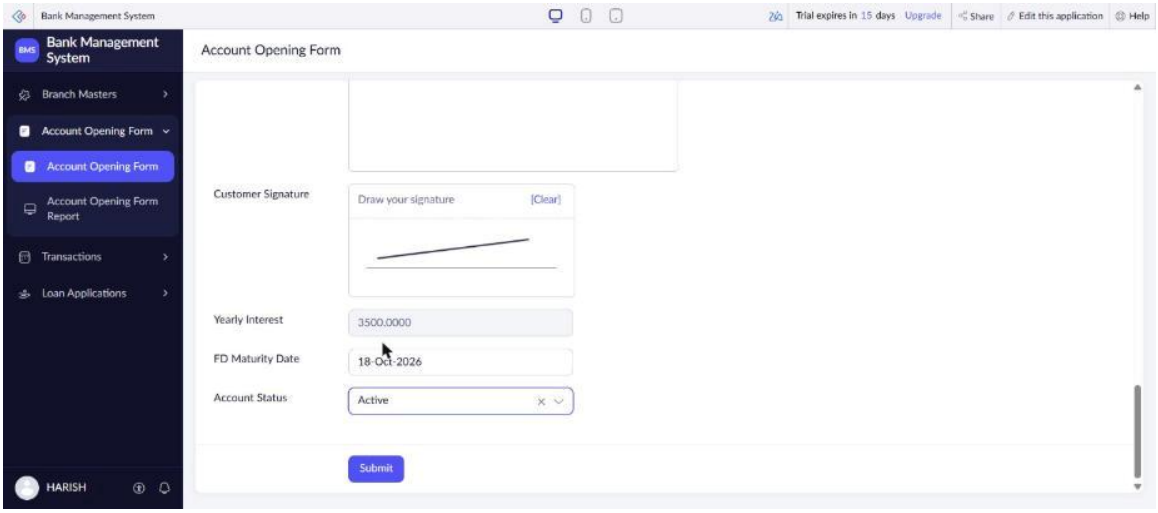


Figure 8.1: Scenario Step 1 – Opening a New Account

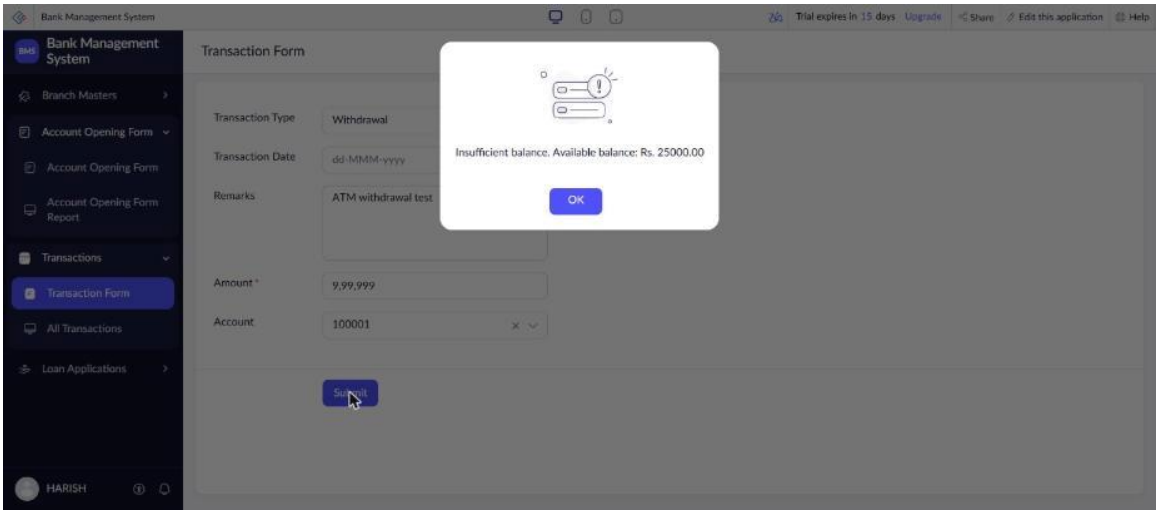


Figure 8.2: Scenario Step 2 – Withdrawal Blocked for Insufficient Balance

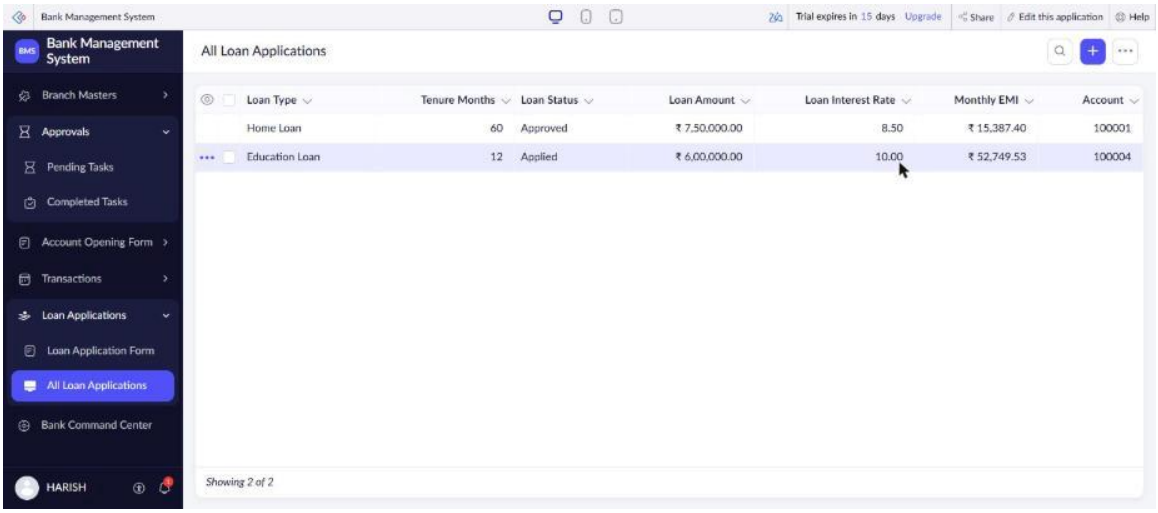


Figure 8.3: Scenario Step 3 – Loan Approved with Calculated EMI

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Linked data model and four forms	Sections 4.2, 4.3, 5.2	Four forms, 3 lookups, 27 field types on Account Opening
Automate balance, withdrawal check and EMI	Sections 5.5, 5.6	TC-06, TC-07, TC-10 passed
Approval and blueprint lifecycle	Section 5.6	TC-12, TC-13 passed; blueprint run passed (TC-15)
Automatic email alerts and FD reminders	Sections 5.6, 5.7	TC-14 passed (2 reminders)
Reports and live dashboard	Sections 5.3, 5.4	Kanban, Calendar, dashboard total ₹4,83,500

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Time to open one account	About 15–20 minutes (paper + Excel)	About 3–5 minutes (one form)	Faster, typed once
Withdrawal check	Manual, can be missed	Automatic alert	No negative balance
EMI calculation	Calculator, error-prone	Automatic on save	Correct and instant
Loan approval	Paper signature	Pending Tasks with comment and time	Traceable and faster

Parameter	Existing System	Proposed Application	Improvement
Branch overview	Monthly manual report	Live dashboard	Always up to date

The time values for the existing system are estimates for the case study, not measured values.

8.4 Limitations of the Low-Code Approach

Some limits of low-code were seen directly. Pivot reports need an extra service (Zoho Analytics). Roles, environments and some features depend on the plan, so the trial could not show them. Drag-and-drop building is quick for a few fields but slow for a large form, and complex banking logic (interest posting, real fund transfer with rollback, links to core banking) would need outside services. The app also depends on one vendor, so moving it to another platform would mean rebuilding it [8].

Workarounds used: a page chart instead of a pivot chart, a schedule plus an edit trigger instead of batch and report workflows, and careful validation scripts to protect data where roles were not available.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

No-code: forms, field properties, reports, the Kanban, the calendar, the dashboard page, the approval and the blueprint were made without writing code. Low-code: the business rules (opening balance, balance check, EMI, reminders, sample data) needed short Deluge scripts. Pro-code: features such as real payment transfer, core banking links or a custom mobile UI would move into pro-code (Catalyst functions or widgets). Most of this app sits in no-code and low-code, which is why it could be built quickly.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

A working Bank Management System was built on Zoho Creator with four linked forms, nine Deluge scripts, four form workflows, a daily schedule, an approval process, a blueprint, six reports and a dashboard. Tests on the live app showed that the main rules work: invalid withdrawals are blocked, balances update, EMI is correct, high-value loans wait for approval and FD reminders are sent. The project shows that a useful banking prototype can be made in days with very little code.

9.2 Future Enhancements

- Create Branch Manager and Cashier roles with record-level access on a paid plan.
- Add a customer portal so customers can see their balance and loan status.
- Use Zia or Zoho Analytics to predict loan default risk and show pivot reports.
- Connect to an SMS gateway and a credit-score API through Connections and invokeurl.
- Use Development, Stage and Production environments for safe updates.

9.3 Learning Outcomes and Career Relevance

This project gave hands-on practice in data modelling, form design, Deluge scripting, workflow automation and testing on a real platform. It also showed the limits of low-code and when pro-code is needed. These skills match roles such as Zoho Creator developer, low-code developer and business process automation specialist, where knowing both the platform and basic programming is valued.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 16 October 2026).

[3] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 16 October 2026).

[4] Zoho Corporation, "Zoho Creator Learning Centre." [Online]. Available: https://www.zoho.com/creator/resources/ (Accessed: 16 October 2026).

[5] Gartner, Inc., "Magic Quadrant for Enterprise Low-Code Application Platforms," Gartner Research, 2024.

[6] A. Sahay, A. Indamutsa, D. Di Ruscio, and A. Pierantonio, "Supporting the understanding and comparison of low-code development platforms," in Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA), 2020, pp. 171–178, doi: 10.1109/SEAA51224.2020.00036.

[7] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, "Low-code development and model-driven engineering: Two sides of the same coin?," Software and Systems Modeling, vol. 21, pp. 437–446, 2022, doi: 10.1007/s10270-021-00970-2.

[8] A. C. Bock and U. Frank, "Low-Code Platform," Business & Information Systems Engineering, vol. 63, pp. 733–740, 2021, doi: 10.1007/s12599-021-00726-8.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Bank Management System app; sections per form + Approvals + page	5.1	Fig 5.1, 5.2
II	Forms with ≥ 8 distinct field types	Y	Account Opening Form: 27 field types	5.2	Fig 5.3, Table 4.2
II	Lookup field (relationship between forms)	Y	Branch Master, Account lookups	4.2, 5.2	Fig 4.2
II	Subform (parent–child data)	Y	Nominee Details subform in Account Opening Form	4.2, 5.2	Fig 5.8, 5.9
II	Field validations / layout rules	Y	Mandatory fields; DS-01, DS-03, DS-04	5.2	Code 5.1–5.3
II	List report with filter, grouping, custom action	Y	List reports with search and actions (no custom grouping)	5.3	Fig 5.13
II	Kanban / Calendar / Timeline / Map report	Y	Account Status Kanban, FD Maturity Calendar	5.3	Fig 5.11, 5.12
II	Pivot table / chart report	N	Needs Zoho Analytics; replaced by page chart	5.3	Fig 5.15
II	Page / dashboard built with UI Builder	Y	Bank Command Center	5.4	Fig 5.16, 5.17
II	Deluge – form events (validate / success / field actions)	Y	DS-01 to DS-04	5.5	Code 5.1–5.3
II	Deluge – functions, fetch, loops, collections	Y	DS-05, DS-07 (fetch, for each, map, list)	5.5	App. C
II	Email / in-app notifications	Y	DS-02, DS-05, approval email	5.5, 5.6	Code 5.4
II	Form workflow	Y	4 form workflows	5.6	Fig 5.18

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Scheduled workflow	Y	Daily FD Maturity Reminder	5.6	Fig 5.25
II	Approval process	Y	High Value Loan Approval	5.6	Fig 5.20, 5.23
II	Blueprint	Y	Account Lifecycle	5.6	Fig 5.21
II	Batch / report workflow	Y	Monthly Savings Interest Credit	5.6	Fig 5.22
II	Testing & debugging evidence	Y	Test cases, defect log, Execute log	6	Table 6.1, Fig 6.1
III	Integration with a Zoho service	Y	Zoho Mail via sendmail	5.7	Code 5.4
III	Connection / REST API call (invokeurl)	Y	invokeurl to Frankfurter API (DS-09)	5.7	Fig 5.26
III	Webhook or Creator REST API	N	Not done (future work)	5.7	—
III	Users, roles and permission profiles	N	Default profiles only; roles not configurable on trial	5.8	Fig 5.27
III	Record-level access criteria / customer portal	Y	Customer portal profile: own records only	5.8	Fig 5.28
III	Zia / AI feature	N	Not used	5.9	—
III	Analytics (charts / Zoho Analytics)	Y	Dashboard panel and chart	5.9	Fig 5.29
IV	Performance optimisation applied & measured	Y	Measured page response times	7.2	Section 7.2
IV	Deployment across environments	N	Environments not on trial	7.1	—
IV	Security controls (encryption, OAuth, permissions)	Y	Login, profiles, validations, approval	7.3	Table 7.3
IV	Governance (audit trail, administration)	Y	Settings page, Added/Modified history, approval history	7.4	Fig 7.1
V	Pro-code extension (Catalyst / widgets / custom functions)	Y	EMI Calculator widget (HTML/JS + Widget SDK)	5.10	Fig 5.30, 5.31

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
V	Low-code limitations & spectrum analysis	Y	Limitations and spectrum discussed	8.4, 8.5	—
—	Mobile app / portal access	Y	Phone/tablet layouts; portal profile	5.11	Fig 5.32

Total features implemented: 27 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	Good	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	Good	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	Satisfactory	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	Satisfactory	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	Good	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Account Opening Form – DS-01 and DS-02

```
// DS-01 | Form: Account Opening | Event: Created -> Validations on form
submission
// Set the opening balance and submission time for a new account
input.Current_Balance = input.Opening_Deposit;
input.Submitted_On = zoho.currenttime;
if(input.Opening_Deposit < 1000)
{
    alert "Minimum opening deposit is Rs. 1000";
    cancel submit;
}

// DS-02 | Form: Account Opening | Event: Created -> Successful form
submission
sendmail
[
    from :zoho.adminuserid
    to :zoho.loginuserid
    subject :"Account Opening Request Received - " + input.Account_Number
    message :"Dear " + input.Customer_Name + ",<br><br>Thank you for your
account opening request.<br>Account Number: " + input.Account_Number +
"<br>Account Type: " + input.Account_Type + "<br>Opening Deposit: Rs. " +
input.Opening_Deposit + "<br><br>Our branch team will verify your KYC and
contact you soon.<br><br>Regards,<br>Bank Team"
]
```

C.2 Transaction Form – DS-03

```
// DS-03 | Form: Transaction | Event: Created -> Validations on form
submission
acc = Account_Opening_Form[ID == input.Account];
if(acc.count() == 0)
{
    alert "Please select a valid account.";
    cancel submit;
}
else
{
    balance = ifnull(acc.Current_Balance,0);
    if(input.Transaction_Type != "Deposit" && input.Amount > balance)
    {
        alert "Insufficient balance. Available balance: Rs. " + balance;
        cancel submit;
    }
    else
    {
        if(input.Transaction_Type == "Deposit")
        {
            acc.Current_Balance = balance + input.Amount;
        }
        else
        {
            acc.Current_Balance = balance - input.Amount;
        }
        if(input.Transaction_Date == null)
        {
            input.Transaction_Date = zoho.currentdate;
        }
    }
}
```

C.3 Loan Application Form – DS-04

```
// DS-04 | Form: Loan Application | Event: Created or Edited ->
Validations
// Calculate monthly EMI using the reducing balance formula
P = ifnull(input.Loan_Amount,0);
yearlyRate = ifnull(input.Loan_Interest_Rate,0);
N = ifnull(input.Tenure_Months,0);
if(P <= 0 || N <= 0)
{
    alert "Loan amount and tenure must be greater than zero.";
    cancel submit;
}
else
{
    R = yearlyRate / 12 / 100;
    if(R == 0)
    {
        input.Monthly_EMI = (P / N).round(2);
    }
    else
    {
        factor = (1 + R).power(N);
        input.Monthly_EMI = (P * R * factor / (factor - 1)).round(2);
    }
    if(input.Loan_Status == null)
    {
        input.Loan_Status = "Applied";
    }
}
```

C.4 Schedule – DS-05

```
// DS-05 | Schedule: Daily FD Maturity Reminder (daily)
todayDate = zoho.currentdate;
endDate = todayDate.addDay(7);
count = 0;
for each acc in Account_Opening_Form[FD_Maturity_Date >= todayDate &&
FD_Maturity_Date <= endDate && Account_Status == "Active"]
{
    sendmail
    [
        from :zoho.adminuserid
        to :zoho.adminuserid
        subject :"FD Maturity Reminder - Account " + acc.Account_Number
        message :"Dear Customer,<br><br>Your Fixed Deposit of Rs. " +
acc.FD_Amount + " in account " + acc.Account_Number + " will mature on "
+ acc.FD_Maturity_Date + ".<br>Please visit the branch to renew or
withdraw.<br><br>Regards,<br>Bank Team"
    ]
    count = count + 1;
}
info "FD reminders sent: " + count;
```

C.5 Approval Action – DS-06

```
// DS-06 | Approval: High Value Loan Approval -> Level 1 -> Approve
// Mark the loan as approved by the branch manager
input.Loan_Status = "Approved";
```

C.6 Function Load_Sample_Data – DS-07 (first version)

```
// DS-07 (first version, run once) | Function: Load_Sample_Data
void Load_Sample_Data()
{
    b1 = Branch_Master[Branch_Name == "Thandalam"].ID;
```

```
    b2 = Branch_Master[Branch_Name == "Poonamallee"].ID;
    b3 = Branch_Master[Branch_Name == "T Nagar"].ID;
    names = {"Arun Kumar","Priya Raman","Karthik S","Divya M","Rahul
V","Sneha R"};
    types = {"Savings","Current","Savings","Fixed
Deposit","Savings","Current"};
    deposits = {25000,120000,8000,200000,15500,60000};
    statusList = {"Active","Approved","KYC
Pending","Active","Applied","Closed"};
    branches = {b1,b2,b1,b3,b2,b3};
    cities =
{"Chennai","Chennai","Kanchipuram","Chennai","Tiruvallur","Chennai"};
    i = 0;
    for each n in names
    {
        parts = n.toList(" ");
        insert into Account_Opening_Form
        [
            Added_User=zoho.loginuser
            Customer_Name.first_name=parts.get(0)
            Customer_Name.last_name=parts.get(1)
            Email_Address=zoho.loginuserid
            Mobile_Number="+91987654321" + i
            PAN_Number="ABCDE123" + i + "F"
            Account_Type=types.get(i)
            Customer_Category="Individual"
            Agree_To_Bank_Terms={"I agree to the bank terms and
conditions"}
            Services_Required={"ATM Card","Net Banking"}
            Opening_Deposit=deposits.get(i)
            Current_Balance=deposits.get(i)
            FD_Amount=deposits.get(i)
            Interest_Rate=7
            Minimum_Balance=1000
            Account_Status=statusList.get(i)
            Branch_Master=branches.get(i)
            FD_Maturity_Date=zoho.currentdate.addDay(2 + i * 3)
            Submitted_On=zoho.currenttime
            Purpose_of_Account="Savings and fixed deposit for " +
cities.get(i) + " branch customer"
        ];
        i = i + 1;
    }
    info "Sample data loaded: 3 branches and " + i + " accounts";
}
```

C.7 Function Load_Sample_Data – DS-07 (current version)

```
// DS-07 (current version) | Function: Load_Sample_Data
void Load_Sample_Data()
{
    // 1. Create branches only if they do not exist
    branchData =
{"Thandalam":"BANK0001234","Poonamallee":"BANK0001235","T
Nagar":"BANK0001236"};
    for each bName in branchData.keys()
    {
        if(Branch_Master[Branch_Name == bName].count() == 0)
        {
            insert into Branch_Master
            [
                Added_User=zoho.loginuser
                Branch_Name=bName
                IFSC_Code=branchData.get(bName)
                City="Chennai"
            ];
        }
```

```
    }
    // 2. Link every account without a branch to a branch (round robin)
    branchIds = Branch_Master[ID != 0].ID.getAll();
    i = 0;
    for each acc in Account_Opening_Form[Branch_Master == null]
    {
        acc.Branch_Master = branchIds.get(i % branchIds.size());
        i = i + 1;
    }
    // 3. Correct the cash deposit entry that was saved as a withdrawal
    for each t in Transaction_Form[Remarks == "Cash deposit at branch" &&
Transaction_Type == "Withdrawal"]
    {
        t.Transaction_Type = "Deposit";
        acc = Account_Opening_Form[ID == t.Account];
        acc.Current_Balance = acc.Current_Balance + 2 * t.Amount;
    }
    info "Branches: " + Branch_Master[ID != 0].count() + ", accounts
linked: " + i;
}
```

C.8 Batch Workflow – DS-08

```
// DS-08 | Batch workflow: Monthly Savings Interest Credit (runs monthly,
1000 records per block)
// Credits monthly interest at 3% per year to every Active Savings
account
if(input.Account_Type == "Savings" && input.Account_Status == "Active")
{
    bal = ifnull(input.Current_Balance,0);
    monthlyInterest = (bal * 3 / 100 / 12).round(2);
    input.Current_Balance = bal + monthlyInterest;
}
```

C.9 Function Get_Exchange_Rates – DS-09

```
void Get_Exchange_Rates()
{
    // DS-09 | REST API call with invokeurl (public Frankfurter API, no
key needed)
    // Gets today's value of 1 INR in USD and EUR for foreign remittance
desk
    response = invokeurl
    [
        url :"https://api.frankfurter.app/latest?from=INR&to=USD,EUR"
        type :GET
    ];
    info "API response: " + response;
    rates = response.get("rates");
    info "1 INR = " + rates.get("USD") + " USD";
    info "1 INR = " + rates.get("EUR") + " EUR";
    info "Rs. 1,00,000 = " + (100000 *
rates.get("USD").toDecimal()).round(2) + " USD";
}
```

C.10 Pro-code Widget – app/widget.html (EMI Calculator)

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>EMI Calculator Widget</title>
<script
src="https://js.zohostatic.com/creator/widgets/version/1.0/widgetsdk-
min.js"></script>
```

```
<style>
  body { font-family: Arial, sans-serif; margin: 0; padding: 16px;
         background: #f5f7fb; color: #1d2433; }
  .card { background: #fff; border-radius: 10px; padding: 16px;
          box-shadow: 0 1px 4px rgba(0,0,0,.08); max-width: 520px; }
  h3 { margin: 0 0 12px; color: #3b2fc9; }
  label { display: block; font-size: 13px; margin-top: 10px; }
  input { width: 100%; padding: 8px; border: 1px solid #c9cfdc;
          border-radius: 6px; box-sizing: border-box; }
  button { margin-top: 14px; background: #3b2fc9; color: #fff;
           border: 0; padding: 9px 16px; border-radius: 6px;
           cursor: pointer; }
  .out { margin-top: 14px; font-size: 15px; }
  table { width: 100%; border-collapse: collapse; margin-top: 10px;
          font-size: 12px; }
  td, th { border-bottom: 1px solid #e3e7ef; padding: 4px;
           text-align: right; }
  th:first-child, td:first-child { text-align: left; }
  #live { margin-top: 12px; font-size: 13px; color: #555; }
</style>
</head>
<body>
<div class="card">
  <h3>Loan EMI Calculator (Pro-code Widget)</h3>
  <label>Loan amount (Rs.)
    <input id="p" type="number" value="600000"></label>
  <label>Interest rate (% per year)
    <input id="r" type="number" step="0.1" value="10"></label>
  <label>Tenure (months)
    <input id="n" type="number" value="12"></label>
  <button onclick="calc()">Calculate</button>
  <div class="out" id="out"></div>
  <table id="sched"></table>
  <div id="live">Loading loan count from the app...</div>
</div>
<script>
function calc() {
  var P = +document.getElementById('p').value;
  var y = +document.getElementById('r').value;
  var N = +document.getElementById('n').value;
  if (P <= 0 || N <= 0) {
    document.getElementById('out').textContent =
      'Enter a valid amount and tenure.';
    return;
  }
  var R = y / 12 / 100;
  var emi = (R === 0) ? P / N
          : P * R * Math.pow(1 + R, N) / (Math.pow(1 + R, N) - 1);
  var total = emi * N;
  document.getElementById('out').innerHTML =
    'Monthly EMI: <b>Rs. ' + emi.toFixed(2) + '</b><br>' +
    'Total interest: Rs. ' + (total - P).toFixed(2);

  var bal = P;
  var rows = '<tr><th>Month</th><th>Interest</th>' +
             '<th>Principal</th><th>Balance</th></tr>';
  for (var m = 1; m <= Math.min(N, 6); m++) {
    var i = bal * R, pr = emi - i;
    bal -= pr;
    rows += '<tr><td>' + m + '</td>' +
            '<td>' + i.toFixed(2) + '</td>' +
            '<td>' + pr.toFixed(2) + '</td>' +
            '<td>' + Math.max(bal, 0).toFixed(2) + '</td></tr>';
  }
  document.getElementById('sched').innerHTML = rows;
}
calc();
```

```
try {
  ZOHO.CREATOR.init().then(function () {
    return ZOHO.CREATOR.API.getAllRecords({
      reportName: "All_Loan_Applications", page: 1, pageSize: 100
    });
  }).then(function (resp) {
    var d = (resp && resp.data) || [];
    var total = d.reduce(function (s, x) {
      return s + (parseFloat(x.Loan_Amount) || 0);
    }, 0);
    document.getElementById('live').textContent =
      'Live from app: ' + d.length + ' loan applications, total Rs. ' +
      total.toFixed(2);
  }).catch(function (e) {
    document.getElementById('live').textContent =
      'Live data not available: ' + JSON.stringify(e).slice(0, 120);
  });
} catch (e) {
  document.getElementById('live').textContent = 'Widget SDK not loaded.';
}
</script>
</body>
</html>
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open https://creatorapp.zoho.in/harish.m.2024.aiml_rajala578/bank-management-system in Chrome and sign in with the Zoho account that has access to the app. The left menu shows Branch Masters, Approvals, Account Opening Form, Transactions, Loan Applications and the Bank Command Center page.

D.2 Instructions for Branch Manager (Administrator)

- Open Bank Command Center to see the total balance and balance by account type.
- Open Approvals → Pending Tasks, click a loan, read the summary and click Approve or Reject with a comment.
- Open Account Status Kanban and drag an account card to change its status, or use the blueprint buttons on a new account.
- Open FD Maturity Calendar to see which fixed deposits mature this month.

D.3 Instructions for Bank Staff

- Open Account Opening Form, fill the customer details, tick the terms and click Submit.
- To record money, open Transaction Form, choose Deposit, Withdrawal or Fund Transfer, enter the amount, pick the account and Submit. If the amount is more than the balance, the app will stop it.
- To apply for a loan, open Loan Application Form, enter type, amount, rate and tenure, pick the account and Submit. The EMI is filled in automatically.

APPENDIX E – EVALUATION RUBRIC

Student Name	Harish M	Register No.	241501066
Project Title	Bank Management System	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

Student Name	Harish M	Register No.	241501066
Department / Section	Artificial Intelligence And Machine Learning	Date of Viva	
Application URL Verified		Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER

Name: _____

EXTERNAL EXAMINER

Name: _____